

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
```

```
# Creating sample agriculture dataset
data = {
    "Rainfall": [800, 600, 750, 900, 650, 700, 850, 720],
    "Temperature": [25, 30, 28, 24, 29, 27, 26, 28],
    "Soil_Nitrogen": [40, 35, 45, 50, 38, 42, 48, 44],
    "Fertilizer": [100, 80, 90, 120, 85, 95, 110, 92],
    "Yield": [30, 22, 28, 35, 24, 27, 33, 29]
}

df = pd.DataFrame(data)
print(df.head())
```

	Rainfall	Temperature	Soil_Nitrogen	Fertilizer	Yield
0	800	25	40	100	30
1	600	30	35	80	22
2	750	28	45	90	28
3	900	24	50	120	35
4	650	29	38	85	24

```
print(df.isnull().sum())
```

```
Rainfall      0
Temperature    0
Soil_Nitrogen  0
Fertilizer     0
Yield          0
dtype: int64
```

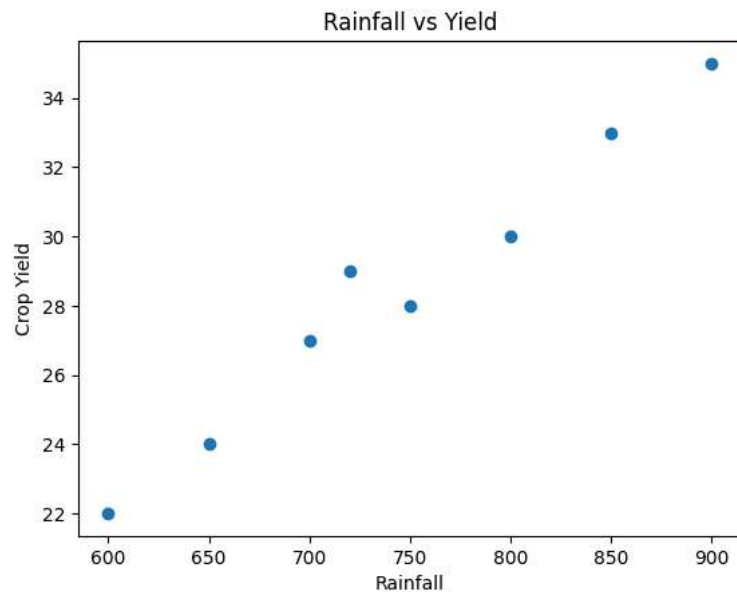
```
X = df.drop("Yield", axis=1)
y = df["Yield"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
print(df.corr())
```

	Rainfall	Temperature	Soil_Nitrogen	Fertilizer	Yield
Rainfall	1.000000	-0.927654	0.878076	0.957662	0.983998
Temperature	-0.927654	1.000000	-0.708885	-0.931253	-0.905858
Soil_Nitrogen	0.878076	-0.708885	1.000000	0.850220	0.915048
Fertilizer	0.957662	-0.931253	0.850220	1.000000	0.958267
Yield	0.983998	-0.905858	0.915048	0.958267	1.000000

```
plt.scatter(df["Rainfall"], df["Yield"])
plt.xlabel("Rainfall")
plt.ylabel("Crop Yield")
plt.title("Rainfall vs Yield")
plt.show()
```



```
print(X.columns)
```

```
Index(['Rainfall', 'Temperature', 'Soil_Nitrogen', 'Fertilizer'], dtype='object')
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42  
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
y_pred = model.predict(X_test)  
print("Predicted Yield:", y_pred)  
df
```

```
Predicted Yield: [22.0969697 28.20557576]
```

	Rainfall	Temperature	Soil_Nitrogen	Fertilizer	Yield
0	800	25	40	100	30
1	600	30	35	80	22
2	750	28	45	90	28
3	900	24	50	120	35
4	650	29	38	85	24
5	700	27	42	95	27
6	850	26	48	110	33
7	720	28	44	92	29

```
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)
```

```
print("Mean Absolute Error:", mae)  
print("R2 Score:", r2)
```

```
Mean Absolute Error: 0.65127272727261  
R2 Score: 0.8829747176492194
```

```
new_data = [[600,29,38,65]]
predicted_yield = model.predict(new_data)

print("Predicted Yield:", predicted_yield)
```

Predicted Yield: [355.33503498]

```
import numpy as np

# Step 1: Agriculture dataset
# Columns: [Soil Nitrogen, Water Usage, Crop Yield]
data = np.array([
    [50, 200, 30],
    [55, 220, 35],
    [80, 300, 60],
    [85, 320, 65],
    [40, 180, 25],
    [90, 350, 70]
])

# Step 2: Choose number of clusters
k = 3

# Step 3: Initialize centroids randomly
np.random.seed(0)
centroids = data[np.random.choice(len(data), k, replace=False)]

def calculate_distance(point, centroids):
    return np.sqrt(((point - centroids) ** 2).sum(axis=1))

# Step 4 & 5: Iterate
for iteration in range(100):
    clusters = []

    for point in data:
        distances = calculate_distance(point, centroids)
        cluster = np.argmin(distances)
        clusters.append(cluster)

    clusters = np.array(clusters)

    old_centroids = centroids.copy()

    # Update centroids
    for i in range(k):
        points = data[clusters == i]
        if len(points) > 0:
            centroids[i] = points.mean(axis=0)

    if np.all(old_centroids == centroids):
        break

print("Final Centroids:")
print(centroids)
print("Cluster Assignment:")
print(clusters)
df
```

```
Final Centroids:
[[ 90 350  70]
 [ 82 310  62]
 [ 48 200  30]]
Cluster Assignment:
[2 2 1 1 2 0]
```

	Rainfall	Fertilizer	Yield
0	100	20	30
1	120	25	35
2	140	30	45
3	160	35	50
4	180	40	60
5	200	45	65

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = {
    "Rainfall": [100,120,140,160,180,200],
    "Fertilizer": [20,25,30,35,40,45],
    "Yield": [30,35,45,50,60,65]
}

df = pd.DataFrame(data)

X = df[["Rainfall","Fertilizer"]]
y = df["Yield"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("Error:", mean_squared_error(y_test, pred))

new = [[170,38]]
print("Predicted Yield:", model.predict(new)[0])

```

```

Error: 3.25
Predicted Yield: 55.04117647058824
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(

```

```

import pandas as pd

data = {
    "Rainfall": [100, 120, 140, 160, 180, 200],
    "Fertilizer": [20, 25, 30, 35, 40, 45],
    "Yield": [30, 35, 45, 50, 60, 65]
}

df = pd.DataFrame(data)
print(df)

```

	Rainfall	Fertilizer	Yield
0	100	20	30
1	120	25	35
2	140	30	45
3	160	35	50
4	180	40	60
5	200	45	65

```
print(df.isnull().sum())
```

```

Rainfall    0
Fertilizer  0
Yield       0
dtype: int64

```

```

from sklearn.model_selection import train_test_split

X = df[["Rainfall","Fertilizer"]]
y = df["Yield"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
df

```


	Rainfall	Fertilizer	Yield
0	100	20	30
1	120	25	35
2	140	30	45
3	160	35	50
4	180	40	60
5	200	45	65

```
from sklearn.metrics import mean_squared_error

predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)

print("Error:", mse)
df
```

Error: 3.25

	Rainfall	Fertilizer	Yield
0	100	20	30
1	120	25	35
2	140	30	45
3	160	35	50
4	180	40	60
5	200	45	65

```
new_data = [[170, 38]] # rainfall, fertilizer
result = model.predict(new_data)

print("Predicted Yield:", result[0])
df
```

Predicted Yield: 55.04117647058824
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn(

	Rainfall	Fertilizer	Yield
0	100	20	30
1	120	25	35
2	140	30	45
3	160	35	50
4	180	40	60
5	200	45	65

```
import numpy as np
import matplotlib.pyplot as plt

class KMeansFromScratch:
    """K-Means clustering implemented from scratch."""

    def __init__(self, n_clusters=3, max_iters=100, random_state=42):
        self.n_clusters = n_clusters
        self.max_iters = max_iters
        self.random_state = random_state
        self.centroids = None
        self.labels = None
        self.inertia = None

    def fit(self, X):
        """Fit K-Means to data X."""
        np.random.seed(self.random_state)
```

```

# Step 1: Initialize centroids randomly
# Pick K random points from data as initial centroids
random_indices = np.random.choice(len(X), self.n_clusters, replace=False)
self.centroids = X[random_indices].copy()

# Iterate until convergence or max iterations
for iteration in range(self.max_iters):
    # Step 2: Assignment - assign each point to nearest centroid
    self.labels = self._assign_clusters(X)

    # Step 3: Update - recalculate centroids
    new_centroids = self._update_centroids(X)

    # Check for convergence (centroids don't change)
    if np.allclose(self.centroids, new_centroids):
        print(f'Converged at iteration {iteration + 1}')
        break

    self.centroids = new_centroids

# Calculate final inertia (within-cluster sum of squares)
self.inertia = self._calculate_inertia(X)

return self

def _assign_clusters(self, X):
    """Assign each point to the nearest centroid."""
    # Calculate distance from each point to each centroid
    distances = np.zeros((len(X), self.n_clusters))

    for k in range(self.n_clusters):
        # Euclidean distance: sqrt(sum((x - centroid)^2))
        distances[:, k] = np.sqrt(np.sum((X - self.centroids[k]) ** 2, axis=1))

    # Assign to nearest centroid (minimum distance)
    return np.argmin(distances, axis=1)

def _update_centroids(self, X):
    """Update centroids as mean of assigned points."""
    new_centroids = np.zeros((self.n_clusters, X.shape[1]))

    for k in range(self.n_clusters):
        # Find all points assigned to cluster k
        cluster_points = X[self.labels == k]

        if len(cluster_points) > 0:
            # Centroid = mean of all points in cluster
            new_centroids[k] = cluster_points.mean(axis=0)
        else:
            # If cluster is empty, keep old centroid
            new_centroids[k] = self.centroids[k]

    return new_centroids

def _calculate_inertia(self, X):
    """Calculate within-cluster sum of squares."""
    inertia = 0
    for k in range(self.n_clusters):
        cluster_points = X[self.labels == k]
        if len(cluster_points) > 0:
            # Sum of squared distances to centroid
            inertia += np.sum((cluster_points - self.centroids[k]) ** 2)
    return inertia

def predict(self, X):
    """Predict cluster for new data."""
    return self._assign_clusters(X)

# Example usage
if __name__ == '__main__':
    # Generate sample data: 3 clusters
    np.random.seed(42)
    cluster1 = np.random.randn(50, 2) + [2, 2]
    cluster2 = np.random.randn(50, 2) + [8, 2]
    cluster3 = np.random.randn(50, 2) + [5, 8]
    X = np.vstack([cluster1, cluster2, cluster3])

```

```

# Fit K-Means
kmeans = KMeansFromScratch(n_clusters=3, max_iters=100)
kmeans.fit(X)

# Results
print(f'\nFinal Inertia: {kmeans.inertia:.2f}')
print(f'\nCentroids:')
for i, centroid in enumerate(kmeans.centroids):
    print(f' Cluster {i}: {centroid}')

# Visualize
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels, cmap='viridis', alpha=0.6)
plt.scatter(kmeans.centroids[:, 0], kmeans.centroids[:, 1],
            c='red', marker='X', s=200, edgecolors='black', linewidths=2)
plt.title('K-Means Clustering (From Scratch)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.colorbar(label='Cluster')
plt.show()

```

Converged at iteration 2

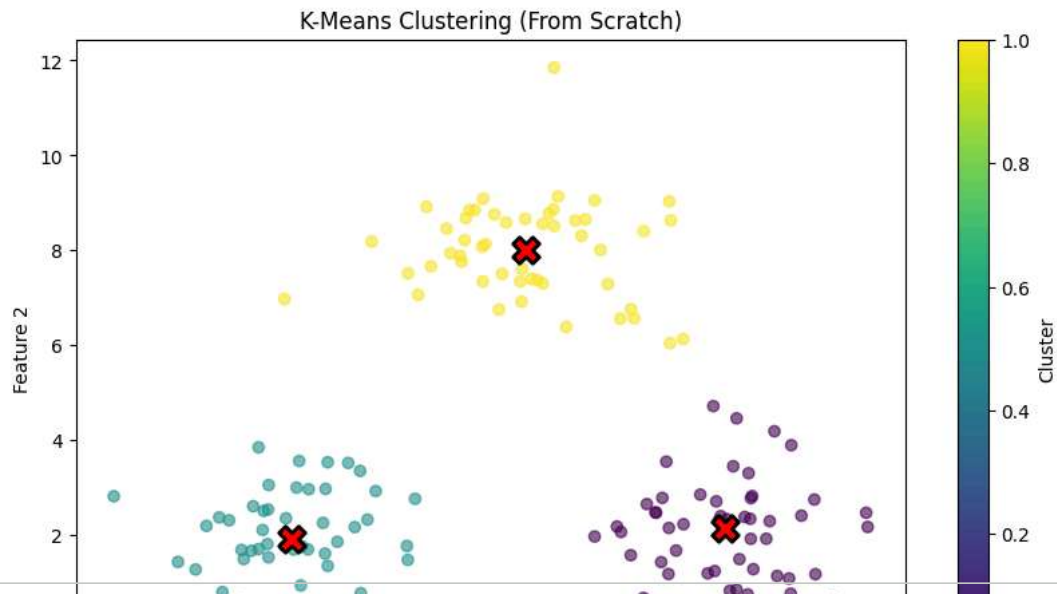
Final Inertia: 286.18

Centroids:

Cluster 0: [7.90454712 2.14006205]

Cluster 1: [1.86432437 1.9279826]

Cluster 2: [5.12894675 8.00084575]



```

import numpy as np

class LogisticRegressionFromScratch:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.lr = learning_rate
        self.iterations = iterations
        self.theta = None

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
        X = np.c_[np.ones(m), X]
        self.theta = np.zeros(n + 1)
        for _ in range(self.iterations):
            h = self.sigmoid(X @ self.theta)
            gradient = (1/m) * X.T @ (h - y)
            self.theta -= self.lr * gradient
        return self

    def predict(self, X):
        m = X.shape[0]

```

```
X = np.c_[np.ones(m), X]
return (self.sigmoid(X @ self.theta) >= 0.5).astype(int)
```

Double-click (or enter) to edit

```
from sklearn.datasets import load_diabetes
import pandas as pd

# Load dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=diabetes.feature_names)
df['progression'] = y

print(df.describe())
```

	age	sex	bmi	bp	s1 \
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	-2.511817e-19	1.230790e-17	-2.245564e-16	-4.797570e-17	-1.381499e-17
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02
min	-1.072256e-01	-4.464164e-02	-9.027530e-02	-1.123988e-01	-1.267807e-01
25%	-3.729927e-02	-4.464164e-02	-3.422907e-02	-3.665608e-02	-3.424784e-02
50%	5.383060e-03	-4.464164e-02	-7.283766e-03	-5.670422e-03	-4.320866e-03
75%	3.807591e-02	5.068012e-02	3.124802e-02	3.564379e-02	2.835801e-02
max	1.107267e-01	5.068012e-02	1.705552e-01	1.320436e-01	1.539137e-01

	s2	s3	s4	s5	s6 \
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	3.918434e-17	-5.777179e-18	-9.042540e-18	9.293722e-17	1.130318e-17
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02
min	-1.156131e-01	-1.023071e-01	-7.639450e-02	-1.260971e-01	-1.377672e-01
25%	-3.035840e-02	-3.511716e-02	-3.949338e-02	-3.324559e-02	-3.317903e-02
50%	-3.819065e-03	-6.584468e-03	-2.592262e-03	-1.947171e-03	-1.077698e-03
75%	2.984439e-02	2.931150e-02	3.430886e-02	3.243232e-02	2.791705e-02
max	1.987880e-01	1.811791e-01	1.852344e-01	1.335973e-01	1.356118e-01

	progression
count	442.000000
mean	152.133484
std	77.093005
min	25.000000
25%	87.000000
50%	140.500000
75%	211.500000
max	346.000000

```
df.isnull()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	progression
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
437	False	False	False	False	False	False	False	False	False	False	False
438	False	False	False	False	False	False	False	False	False	False	False
439	False	False	False	False	False	False	False	False	False	False	False
440	False	False	False	False	False	False	False	False	False	False	False
441	False	False	False	False	False	False	False	False	False	False	False

442 rows × 11 columns

```
from sklearn.datasets import fetch_california_housing
import pandas as pd
```

```
# Load dataset
housing = fetch_california_housing()
X = housing.data
y = housing.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=housing.feature_names)
df['MedHouseVal'] = y

print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
8   MedHouseVal     20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
None
```

```
df.isnull()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
20635	False	False	False	False	False	False	False	False	False
20636	False	False	False	False	False	False	False	False	False
20637	False	False	False	False	False	False	False	False	False
20638	False	False	False	False	False	False	False	False	False
20639	False	False	False	False	False	False	False	False	False

20640 rows × 9 columns

```
from sklearn.datasets import load_wine
import pandas as pd

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=wine.feature_names)
df['cultivar'] = wine.target_names[y]

print(df.describe())
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium \
count	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573
std	0.811827	1.117146	0.274344	3.339564	14.282484
min	11.030000	0.740000	1.360000	10.600000	70.000000
25%	12.362500	1.602500	2.210000	17.200000	88.000000
50%	13.050000	1.865000	2.360000	19.500000	98.000000
75%	13.677500	3.082500	2.557500	21.500000	107.000000
max	14.830000	5.800000	3.230000	30.000000	162.000000

	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins \
--	---------------	------------	----------------------	-------------------

count	178.000000	178.000000	178.000000	178.000000
mean	2.295112	2.029270	0.361854	1.590899
std	0.625851	0.998859	0.124453	0.572359
min	0.980000	0.340000	0.130000	0.410000
25%	1.742500	1.205000	0.270000	1.250000
50%	2.355000	2.135000	0.340000	1.555000
75%	2.800000	2.875000	0.437500	1.950000
max	3.880000	5.080000	0.660000	3.580000

	color_intensity	hue	od280/od315_of_diluted_wines	proline
count	178.000000	178.000000	178.000000	178.000000
mean	5.058090	0.957449	2.611685	746.893258
std	2.318286	0.228572	0.709990	314.907474
min	1.280000	0.480000	1.270000	278.000000
25%	3.220000	0.782500	1.937500	500.500000
50%	4.690000	0.965000	2.780000	673.500000
75%	6.200000	1.120000	3.170000	985.000000
max	13.000000	1.710000	4.000000	1680.000000

```
df.isnull().sum()
```

```

0
MedInc    0
HouseAge  0
AveRooms  0
AveBedrms 0
Population 0
AveOccup  0
Latitude  0
Longitude 0
MedHouseVal 0

```

```
dtype: int64
```

```
df.isnull()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
173	False	False	False	False	False	False	False	False	False
174	False	False	False	False	False	False	False	False	False
175	False	False	False	False	False	False	False	False	False
176	False	False	False	False	False	False	False	False	False
177	False	False	False	False	False	False	False	False	False

```
178 rows × 14 columns
```

```
df.isnull().sum()
```

	0
alcohol	0
malic_acid	0
ash	0
alcalinity_of_ash	0
magnesium	0
total_phenols	0
flavanoids	0
nonflavanoid_phenols	0
proanthocyanins	0
color_intensity	0
hue	0
od280/od315_of_diluted_wines	0
proline	0
cultivar	0

dtype: int64

```
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = iris.target_names[y]

print(df.head())
print(f"\nShape: {df.shape}")
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	species
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa

Shape: (150, 5)

df.isnull()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
145	False	False	False	False	False
146	False	False	False	False	False
147	False	False	False	False	False
148	False	False	False	False	False
149	False	False	False	False	False

150 rows × 5 columns

```
df.isnull().sum()
```

```

      0
sepal length (cm)  0
sepal width (cm)  0
petal length (cm)  0
petal width (cm)  0
species          0

```

dtype: int64

```

import pandas as pd

import numpy as np

farm_data = {

"Crop": ["Wheat", "Rice", "Maize", "Millet", "Pulses"],
"Yield_kg": [3200, np.nan, 3500, 3000, np.nan],
"Water_liters": [4200, 5500, np.nan, 3000, 2500],
"Cost_INR": [18000, 20000, 17000, np.nan, 12000]

}

df = pd.DataFrame(farm_data)

df

```

```

      Crop  Yield_kg  Water_liters  Cost_INR
0  Wheat     3200.0         4200.0    18000.0
1   Rice        NaN         5500.0    20000.0
2  Maize     3500.0          NaN    17000.0
3  Millet     3000.0         3000.0         NaN
4  Pulses        NaN         2500.0    12000.0

```

```

df.fillna(method="ffill", inplace=True)
df

```



```
/tmp/ipython-input-2541247499.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version
df.fillna(method="ffill", inplace=True)
```

	Crop	Yield_kg	Water_liters	Cost_INR
0	Wheat	3200.0	4200.0	18000.0
1	Rice	3200.0	5500.0	20000.0
2	Maize	3500.0	5500.0	17000.0
3	Millet	3000.0	3000.0	17000.0
4	Pulses	3000.0	2500.0	12000.0

```
df.fillna(method="bfill", inplace=True)
df
```

```
/tmp/ipython-input-3840312381.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version
df.fillna(method="bfill", inplace=True)
```

	Crop	Yield_kg	Water_liters	Cost_INR
0	Wheat	3200.0	4200.0	18000.0
1	Rice	3200.0	5500.0	20000.0
2	Maize	3500.0	5500.0	17000.0
3	Millet	3000.0	3000.0	17000.0
4	Pulses	3000.0	2500.0	12000.0

```
import numpy as np

class LogisticRegressionFromScratch:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.lr = learning_rate
        self.iterations = iterations
        self.theta = None

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
        X = np.c_[np.ones(m), X]
        self.theta = np.zeros(n + 1)
        for _ in range(self.iterations):
            h = self.sigmoid(X @ self.theta)
            gradient = (1/m) * X.T @ (h - y)
            self.theta -= self.lr * gradient
        return self

    def predict(self, X):
        m = X.shape[0]
        X = np.c_[np.ones(m), X]
        return (self.sigmoid(X @ self.theta) >= 0.5).astype(int)
```

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 5])

theta0 = 0
theta1 = 0
learning_rate = 0.01
epochs = 1000
n = len(X)

for _ in range(epochs):
    y_pred = theta0 + theta1 * X

    d_theta0 = (-2/n) * np.sum(y - y_pred)
    d_theta1 = (-2/n) * np.sum(X * (y - y_pred))
```

```

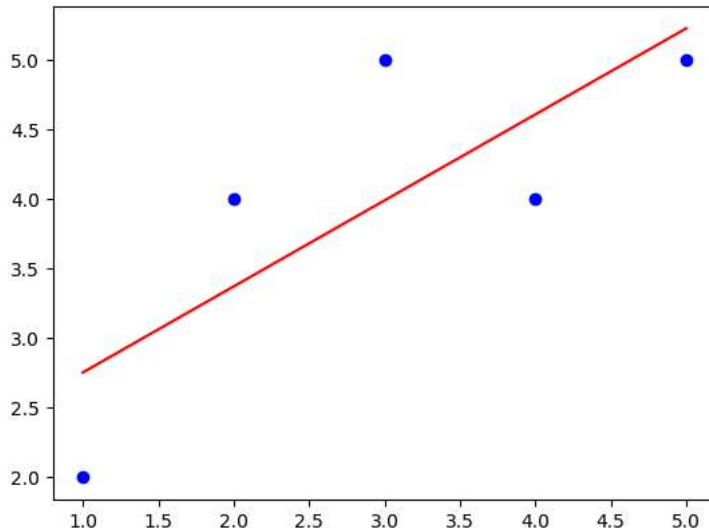
theta0 -= learning_rate * d_theta0
theta1 -= learning_rate * d_theta1

print("Theta0 (Intercept):", theta0)
print("Theta1 (Slope):", theta1)

plt.scatter(X, y, color='blue')
plt.plot(X, theta0 + theta1 * X, color='red')
plt.show()

```

Theta0 (Intercept): 2.136116825825789
 Theta1 (Slope): 0.6176946148762643



```

import numpy as np

X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 5])

x_mean = np.mean(X)
y_mean = np.mean(y)

theta1 = np.sum((X - x_mean) * (y - y_mean)) / np.sum((X - x_mean)**2)
theta0 = y_mean - theta1 * x_mean

print("Theta0 (Intercept):", theta0)
print("Theta1 (Slope):", theta1)

```

Theta0 (Intercept): 2.2
 Theta1 (Slope): 0.6

```

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Sample data
X = np.array([1000, 1500, 2000, 2500, 3000]).reshape(-1, 1) # reshape for sklearn
y = np.array([150000, 200000, 280000, 350000, 420000])

# Create and train model
model = LinearRegression()
model.fit(X, y)

# Make predictions
y_pred = model.predict(X)

# Extract parameters
intercept = model.intercept_
slope = model.coef_[0]

print(f'Intercept: {intercept:.2f}')

```

```

print(f'Slope: {slope:.2f}')
print(f'\nPredictions:')
for i in range(len(X)):
    print(f' X={X[i][0]}: Predicted={y_pred[i]:.0f}, Actual={y[i]}')

# Evaluate model
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)

print(f'\nModel Evaluation:')
print(f' MSE: {mse:.2f}')
print(f' RMSE: {rmse:.2f}')
print(f' R2 Score: {r2:.4f}')

# Visualize
plt.scatter(X, y, color='blue', label='Actual data')
plt.plot(X, y_pred, color='red', label='Fitted line')
plt.xlabel('Square Feet')
plt.ylabel('Price ($)')
plt.legend()
plt.show()

```

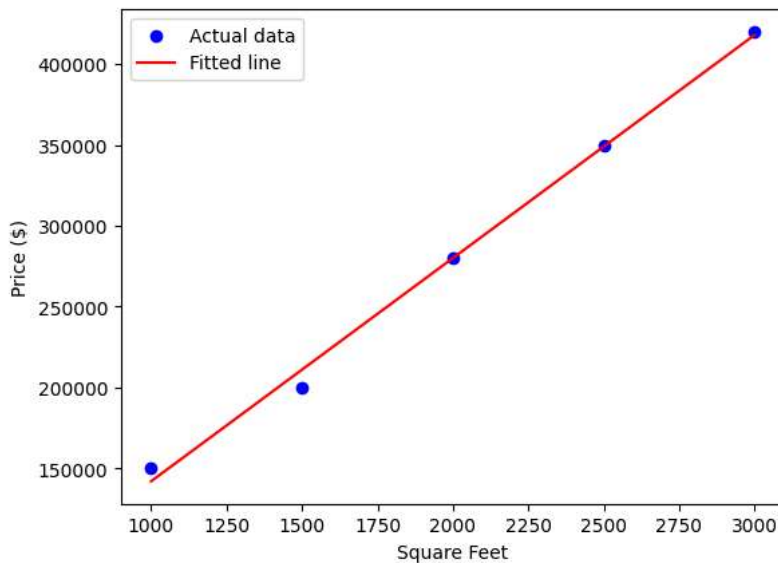
Intercept: 4000.00
Slope: 138.00

Predictions:

X	Predicted	Actual
1000	142000	150000
1500	211000	200000
2000	280000	280000
2500	349000	350000
3000	418000	420000

Model Evaluation:

MSE	3800000.00
RMSE	6164.41
R ² Score	0.9960



Start coding or [generate](#) with AI.

```

from sklearn.datasets import load_wine

data = load_wine()
X = data.data
y = data.target

```

```

print(f"Shape: {X.shape}")
print(f"Features: {data.feature_names}")
print(f"Classes: {data.target_names}")

```

```

Shape: (178, 13)
Features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenol']
Classes: ['class_0' 'class_1' 'class_2']

```

```
from sklearn.datasets import load_iris
```

```
# Load dataset
data = load_iris()
X = data.data # Features
y = data.target # Labels

print(f"Shape: {X.shape}")
print(f"Features: {data.feature_names}")
print(f"Classes: {data.target_names}")
```

```
Shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Classes: ['setosa' 'versicolor' 'virginica']
```

```
rows = int(input("Enter number of rows for half diamond: "))
for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
for i in range(rows - 1, 0, -1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

```
Enter number of rows for half diamond: 5
```

```
  *
 ***
*****
*****
*****
*****
  ***
  *
```

```
N = int(input("Enter N: "))
total = 0
for i in range(1, N + 1):
    total = total + i
print("Sum =", total)
```

```
Enter N: 5
Sum = 15
```

✓ Welcome to Colab!

```
# 1. Simple IF statement
print("=== Simple IF ===")
soil_moisture = 25 # percentage

if soil_moisture < 30:
    print("🚨 Soil is dry! Irrigation needed.")
```

```
=== Simple IF ===
🚨 Soil is dry! Irrigation needed.
```

```
# 1. Simple IF statement
print("=== Simple IF ===")
soil_moisture = 40 # percentage

if soil_moisture < 30:
    print("🚨 Soil is dry! Irrigation needed.")
```

```
=== Simple IF ===
```

```
print("\n=== IF-ELSE ===")
disease_incidence = 57 # percentage

if disease_incidence > 55:
    print("disease alert!")
    print("  take up spraying of pesticides.")
else:
    print("normal.")
```

```
=== IF-ELSE ===
disease alert!
    take up spraying of pesticides.
```

```
print("\n=== IF-ELSE ===")
disease_incidence=57

if disease_incidence > 55:
    print("disease alert!")
    print("    take up spraying of pesticides.")
else:
    print("normal.")
```

```
=== IF-ELSE ===
disease alert!
    take up spraying of pesticides.
```

```
print("\n IF-ELSE ")
disease_incidence=57

if disease_incidence > 55:
    print("disease alert!")
    print("    take up spraying of pesticides.")
else:
    print("normal.")
```

```
IF-ELSE
disease alert!
    take up spraying of pesticides.
```

```
print("\nIF-ELSE ")
disease_incidence=57

if disease_incidence > 55:
    print("disease alert!")
    print("    take up spraying of pesticides.")
else:
    print("normal.")
```

```
IF-ELSE
disease alert!
    take up spraying of pesticides.
```

```
# Demonstrating ML Paradigms with Agricultural Examples

# 1. SUPERVISED LEARNING - Crop Classification
print("=== Supervised Learning Example ===")

# Training data: [temperature, rainfall] -> crop_suitable
training_data = [
    ([25, 100], "rice"),      # Warm, wet = rice
    ([30, 50], "wheat"),      # Warm, moderate = wheat
    ([20, 150], "rice"),      # Cool, very wet = rice
    ([35, 30], "millet"),     # Hot, dry = millet
]

def simple_crop_classifier(temp, rain):
    """Simple supervised learning model"""
    if rain > 100:
        return "rice"
    elif temp > 32 and rain < 40:
        return "millet"
    else:
        return "wheat"

# Test prediction
print(f"Temp=28, Rain=120mm -> {simple_crop_classifier(28, 120)}")

# 2. UNSUPERVISED LEARNING - Soil Clustering
print("\n=== Unsupervised Learning Example ===")
```

```

soil_samples = [
    {"pH": 6.5, "nitrogen": 80, "organic": 3.2},
    {"pH": 6.8, "nitrogen": 75, "organic": 3.0},
    {"pH": 5.2, "nitrogen": 40, "organic": 1.5},
    {"pH": 5.0, "nitrogen": 35, "organic": 1.2},
    {"pH": 7.5, "nitrogen": 90, "organic": 4.0},
]

def simple_soil_clustering(samples):
    """Group soils by pH level (simple clustering)"""
    clusters = {"acidic": [], "neutral": [], "alkaline": []}

    for i, sample in enumerate(samples):
        if sample["pH"] < 6.0:
            clusters["acidic"].append(f"Sample_{i}")
        elif sample["pH"] < 7.2:
            clusters["neutral"].append(f"Sample_{i}")
        else:
            clusters["alkaline"].append(f"Sample_{i}")

    return clusters

clusters = simple_soil_clustering(soil_samples)
print("Soil Clusters:", clusters)

# 3. REINFORCEMENT LEARNING - Irrigation Simulation
print("\n=== Reinforcement Learning Concept ===")

class SimpleIrrigationAgent:
    """Learns irrigation timing through rewards"""

    def __init__(self):
        self.q_values = {"morning": 0, "afternoon": 0, "evening": 0}
        self.learning_rate = 0.1

    def get_action(self):
        # Choose best known action
        return max(self.q_values, key=self.q_values.get)

    def learn(self, action, reward):
        # Update knowledge based on reward
        self.q_values[action] += self.learning_rate * reward
        print(f" Irrigated in {action}: reward = {reward}")

# Simulate learning
agent = SimpleIrrigationAgent()
rewards = {"morning": 10, "afternoon": -5, "evening": 8}

print("Learning optimal irrigation time...")
for episode in range(5):
    for time_slot, reward in rewards.items():
        agent.learn(time_slot, reward)

print(f"Best learned time: {agent.get_action()}")

```

```

=== Supervised Learning Example ===
Temp=28, Rain=120mm -> rice

```

```

=== Unsupervised Learning Example ===
Soil Clusters: {'acidic': ['Sample_2', 'Sample_3'], 'neutral': ['Sample_0', 'Sample_1'], 'alkaline': ['Sample_4']}

```

```

=== Reinforcement Learning Concept ===
Learning optimal irrigation time...
Irrigated in morning: reward = 10
Irrigated in afternoon: reward = -5
Irrigated in evening: reward = 8
Irrigated in morning: reward = 10
Irrigated in afternoon: reward = -5
Irrigated in evening: reward = 8
Irrigated in morning: reward = 10
Irrigated in afternoon: reward = -5
Irrigated in evening: reward = 8
Irrigated in morning: reward = 10
Irrigated in afternoon: reward = -5
Irrigated in evening: reward = 8
Irrigated in morning: reward = 10
Irrigated in afternoon: reward = -5

```

```
Irrigated in evening: reward = 8
Best learned time: morning
```

```
# 1. FOR loop with range
print("=== harshitha 5 Plots ===")
for plot_num in range(1, 6):
    print(f"harshitha Plot {plot_num}...")
```

```
=== harshitha 5 Plots ===
harshitha Plot 1...
harshitha Plot 2...
harshitha Plot 3...
harshitha Plot 4...
harshitha Plot 5...
```

```
# print numbers from 1 TO 100
for i in range (1,15):
    print (i)
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
names=["savitri","harshitha","pruthvi","pavan"]
for n in names:
    print("hello",n)
```

```
hello savitri
hello harshitha
hello pruthvi
hello pavan
```

```
s:
    print(names)names = ["fall army worm","girllder"]
for n in name
```

```
File "/tmp/ipython-input-687544498.py", line 1
  s:
  ^
SyntaxError: invalid syntax
```

```
#print numbers from 1 to 100
for n in range (1 ,20):
    print(n)
```

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

```
marks = int(input("Enter marks: "))
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

```
height = int(input("enter crop height in cm:"))
if height >= 180:
```

```
print("tall")
elif height >=160:
    print("medium")
else:
    print("dwarf")
```

```
yield = int(input("enter crop yield in kg:"))
if yield >=2000:
    print("high yield")
elif yield >=1000:
    print("medium yield")
else :
    print("low")
```

```
crop_yield = int(input("enter crop yield in kg:"))
if crop_yield >=2000:
    print("high yield")
elif crop_yield >=1000:
    print("medium yield")
else :
    print("low")
```

```
for i in range(3): # Outer loop
    for j in range(3): # Inner loop
        print(f"({i},{j})")
```

```
for i in range(1,4):
    print(i)
```

```
for i in range(1,6):
    for j in range(1,6):
        print(i*j,end=" ")
        print(i,j)
```

```
print('Original Image (Pixel Intensities):')
# Simulate a 3x4 grayscale image (0=black, 255=white)
image = [
    [0, 50, 100, 150],
    [200, 255, 200, 150],
    [100, 50, 0, 0]
]

for row in image:
    print(row)

max_intensity = 255 # Assuming 8-bit grayscale image
inverted_image = []

print('\nInverted Image (Pixel Intensities):')
# Outer loop iterates through rows
for r_idx in range(len(image)):
    new_row = []
    # Inner loop iterates through columns (pixels in the current row)
    for c_idx in range(len(image[r_idx])):
        original_pixel_value = image[r_idx][c_idx]
        inverted_pixel_value = max_intensity - original_pixel_value
        new_row.append(inverted_pixel_value)
    inverted_image.append(new_row)

for row in inverted_image:
    print(row)
```

```
print('5x5 Square Pattern:')
for i in range(5): # Outer loop for rows
    for j in range(5): # Inner loop for columns in each row
        print('* ', end='') # Print an asterisk followed by a space
    print() # Move to the next line after each row is complete
```

```
print(" ", end=" ")
for col_num in range(1, 11):
    print(f"{col_num:>4}", end=" ")
```



```

print() # New line after header

print("----" * 11)

for row_num in range(1, 11): # Outer loop for rows
    print(f"{row_num:<2}| ", end="")
    for col_num in range(1, 11): # Inner loop for columns
        product = row_num * col_num
        print(f"{product:>4}", end="")
    print() # New line after each row

```

```

rows = 7 # You can change this to adjust the size of the diamond

print("\nPlough/Diamond Pattern:")

# Upper half of the diamond
for i in range(rows):
    # Print leading spaces
    for j in range(rows - i - 1):
        print(" ", end="")
    # Print stars
    for k in range(2 * i + 1):
        print("*", end="")
    print() # Move to the next line

# Lower half of the diamond
for i in range(rows - 2, -1, -1):
    # Print leading spaces
    for j in range(rows - i - 1):
        print(" ", end="")
    # Print stars
    for k in range(2 * i + 1):
        print("*", end="")
    print() # Move to the next line

```

```

print("\nPen Shape Pattern:")

height = 15
cap_height = 3
cap_width = 7
body_width = 5
tip_height = 3

def print_centered(text, total_width):
    padding = (total_width - len(text)) // 2
    print(" " * padding + text + " " * padding)

for row in range(height):
    if row < cap_height: # Cap of the pen
        if row == 0:
            print_centered("___", cap_width + 2) # Top of the cap
        elif row == cap_height - 1:
            print_centered("|__|", cap_width + 2) # Bottom of the cap
        else:
            print_centered("|  |", cap_width + 2)
    elif row < height - tip_height: # Body of the pen
        print_centered("|  |", cap_width + 2) # Use cap_width for centering relative to widest part
    else: # Tip of the pen
        tip_row = row - (height - tip_height)
        current_tip_width = body_width - (tip_row * 2)
        if current_tip_width < 1: # Ensure tip doesn't go negative width
            current_tip_width = 1

        leading_spaces = (cap_width + 2 - current_tip_width) // 2

```

```

def multi(num1,num2):
    return num1 * num2
result = multi(5,25)
print(result)

```

```

def div(num1,num2):
    return num1 / num2
result = div(125,25)
print(result)

```

```
def say_hello():  
    print("Hello, Welcome to Python!")  
say_hello()
```

```
def greet(shahi):  
    print("Hello", shahi)  
greet("shahi")
```

```
def greet(chatter_box):  
    print("Hello", chatter_box, "have a nice day")  
greet("chatter_box")
```

```
def add(a, b):  
    print("Sum =", a + b)  
add(5, 3)
```

```
def value(a, b):  
    return a + b, a - b, a * b  
print("value",10, 5)
```

```
def value(num):  
    if num % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"  
value(6)
```

```
students = []  
students.append("savitri")  
students.append("shahistha")  
students.append("pruthvi")  
students.append("pavan")  
students.append("harshitha")  
students.sort()  
print(students)
```

```
students = []  
students.append("harshitha")  
students.append("shahistha")  
students.append("pruthvi")  
students.append("pavan")  
students.append("savitri")  
students.remove("harshitha")  
print(students)  
students.sort()  
print(students)
```

```
coordinates = (30, 20,40,25,50)  
print(coordinates[4])
```

```
numbers = {1, 2, 3, 3, 2}  
print(numbers)
```

```
def calculator(a, b):  
    return a + b, a - b, a * b  
print(calculator(10,20))  
print(calculator(25,26))  
print(calculator(35,36))  
print(calculator(25,46))  
print(calculator(34,25))  
print(calculator(25,56))  
print(calculator(60,35))
```

```
def calculator(a, b):  
    return a + b  
print(calculator(10,20))
```

```
def calculator(a, b):
    return a - b, a * b
print(calculator(10,20))
```

```
def calculator(a, b):
    return a + b, a - b, a * b, a / b
result_add, result_sub, result_mul, result_div = calculator(15, 20)
print(f"Addition: {result_add}")
print(f"Subtraction: {result_sub}")
print(f"Multiplication: {result_mul}")
print(f"Division: {result_div}")
```

```
a=input("enter the num1:")
b=input("enter the num2:")
c= a+b
print (c)
```

```
# Get input from the user for the first number
num1_str = input("Enter the first number: ")
# Get input from the user for the second number
num2_str = input("Enter the second number: ")

# Convert the input strings to integers
num1 = int(num1_str)
num2 = int(num2_str)

# Calculate the sum
sum_result = num1 + num2

# Print the result
print(f"The sum of {num1} and {num2} is: {sum_result}")
```

```
def calculator(a, b):
    return a + b, a - b, a * b, a/b
print(calculator(10,30))
print(calculator(25,26))
print(calculator(35,36))
print(calculator(25,46))
print(calculator(34,25))
print(calculator(25,56))
print(calculator(60,35))
```

```
def calculator(a, b,c,d):
    return a - b+c*d, a * b*c/d
print(calculator(10,20,30,50))
```

```
def calculator(a, b):
    return a + b
print(calculator(10,20))
```

```
def calculator(a, b,c,d):
    return a + b-c*d, a * b+c/d
print(calculator(10,20,30,50))
```

```
def calculator(a, b):
    return a + b, a - b, a * b, a/b
print(calculator(10,30))
print(calculator(25,42))
print(calculator(35,36))
print(calculator(26,46))
print(calculator(34,85))
print(calculator(625,56))
print(calculator(160,35))
```

```
crop_name = " wheat "  
print(f"Original: '{crop_name}'")  
print(f"Stripped: '{crop_name.strip()}'")  
print(f"Upper: '{crop_name.strip().upper()}'")  
print(f"Lower: '{crop_name.strip().lower()}'")
```

```
animal_name = "elephant"  
print(f"animal name: '{animal_name}'")
```

```
word = "shahistha"  
print(word[-1])  
print(word[-2])  
print(word[-3])
```

```
name = "savitri"  
print(name[0:5])
```

```
text = "agriinformation "  
print(text[0:8])
```

```
text = "agri information "  
print(text[0:9])
```

```
name= "agriculture is backbone of india"  
print(len(name))
```

```
text = "university of agricultural sciences bangalore"  
text = text.title()  
print(text)
```

```
text = "university of agricultural sciences bangalore"  
text = text.upper()  
print(text)
```

```
text="agriinformation "  
print(text[0:10])
```

```
msg = "I like AI ML"  
print(msg.replace("AI ML", "Agriculture"))
```

```
email = "user@gmail.com"  
print("@ " in email)
```

True

```
words = ["Python", "fun", "is"]  
print(" ".join(words))
```

Python fun is

```
first = "chatter box"  
second = "savitri"  
print(first + " " + second)
```

chatter box savitri

```
name = "savitri"  
address = 'belgavi'  
print(f"My name is {name} and I am from {address} district")
```

My name is savitri and I am from belgavi district

```
text = "variables"  
print(text.count("a"))
```

2

```
file=open("/content/hiiii.txt","r")  
content=file.read()  
print(content)
```

```
-----  
FileNotFoundError                                Traceback (most recent call last)  
/tmp/ipython-input-2429735646.py in <cell line: 0>()  
----> 1 file=open("/content/hiiii.txt","r")  
      2 content=file.read()  
      3 print(content)  
  
FileNotFoundError: [Errno 2] No such file or directory: '/content/hiiii.txt'
```

```
content=file.read()
```

```
print(content)
```

```
file=open("/content/hiiii.txt","w")  
file.write("i am writing python code!")  
file.close()
```

```
entry=input("write python code!")  
with open("/content/hiiii.txt","w") as f:  
    f.write(entry+"\n")
```

```
a = 10  
b = 3  
print(a + b)  
print(a / b)  
print(a % b)  
print(a ** b)
```

```
a = 10  
b = 5  
print(a + b)  
print(a / b)  
print(a % b)  
print(a ** b)
```

```
price = 500  
quantity = 46  
total = price * quantity  
print(total)
```

```
age = 18  
print(age >= 18)
```

```
marks = 75  
attendance = True  
result = marks > 80 and attendance  
print(result)
```

```
x = 10  
x+=50  
print(x)
```

```
students = ["savitri", "harshita", "shahistha"]  
print("savitri" in students)  
print("chatter box" in students)
```

```
a = 90
b = 10
print(a is b)
```

```
x = [1, 6, 3]
y = [1, 5, 4]
print(x is y)
```

```
num = 9
# Break it down:
# Condition 1: Is num positive?
is_positive = num > 0
# Condition 2: Is num even?
is_even = num % 2 == 0
# Combine both conditions
result = is_positive and is_even
print(result)
```

```
import numpy as np

crops = ["Wheat", "Rice", "Maize", "Barley", "Sorghum"]
rainfall_levels = [400, 600, 800, 1000, 1200] # Rainfall in mm
yield_matrix = []

for crop in crops:
    current_crop_yields = []
    for rainfall in rainfall_levels:
        simulated_yield = round((rainfall / 10) + (len(crop) * 5) + np.random.randint(-10, 10), 2)
        current_crop_yields.append(simulated_yield)
    yield_matrix.append(current_crop_yields)

print(f"{'Crop':<10} | {' | '.join(map(str, rainfall_levels))} (mm)")
print("-" * (10 + 3 * len(rainfall_levels)))

for i, crop in enumerate(crops):
    yields = " | ".join(map(str, yield_matrix[i]))
    print(f"{crop:<10} | {yields}")
```

```
import numpy as np

crops = ["Wheat", "Rice", "Maize", "Barley", "Sorghum"]
rainfall_levels = [400, 600, 800, 1000, 1200] # Rainfall in mm
yield_matrix = []

for crop in crops:
    current_crop_yields = []
    for rainfall in rainfall_levels:
        simulated_yield = round((rainfall / 10) + (len(crop) * 5) + np.random.randint(-10, 10), 2)
        current_crop_yields.append(simulated_yield)
    yield_matrix.append(current_crop_yields)

print(f"{'Crop':<10} | {' | '.join(map(str, rainfall_levels))} (mm)")
print("-" * (10 + 3 * len(rainfall_levels)))

for i, crop in enumerate(crops):
    yields = " | ".join(map(str, yield_matrix[i]))
    print(f"{crop:<10} | {yields}")
```

```
crops = ["Wheat", "Rice", "Maize", "Barley", "Soybean"]
rainfall = [200, 400, 600, 800, 1000]
crop_factor = {
    "Wheat": 1.2,
    "Rice": 1.5,
    "Maize": 1.3,
    "Barley": 1.1,
    "Soybean": 1.4
}

yield_matrix = []

for crop in crops:
    row = []
    for rain in rainfall:
```

```

        yield_value = crop_factor[crop] * rain
        row.append(yield_value)
    yield_matrix.append(row)

print("Yield Prediction Matrix (rows = crops, columns = rainfall levels):\n")

for i in range(len(crops)):
    print(crops[i], ":", yield_matrix[i])

```

```

crops=["wheat","corn","soybean"]
rainfall_levels=[50,100,150,200]
crop_factors={
    "wheat":2.5,
    "corn":3.0,
    "soybean":2.0
}
yield_matrix=[]

for crop_name in crops:
    row=[]
    for rainfall in rainfall_levels:
        yield_value=crop_factors[crop_name]*rainfall
        row.append(yield_value)
    yield_matrix.append(row)

print("yield prediction matrix(rows=crops,columns=rainfall levels):\n")

for i in range(len(crops)):
    print((crops[i]),":",yield_matrix[i])

```

```

print('6x6 Sprinkler Coverage Pattern:')

size = 6
center_row = 2
center_col = 2
coverage_radius = 2 # Adjust this value to change the coverage area

for i in range(size):
    for j in range(size):
        # Calculate Manhattan distance from the center for a diamond-like shape
        distance = abs(i - center_row) + abs(j - center_col)
        if i == center_row and j == center_col:
            print("S", end=" ") # Sprinkler location
        elif distance <= coverage_radius:
            print("*", end=" ") # Covered area
        else:
            print(".", end=" ") # Dry area
    print()

```

```

scalar_value = 5
print("SCALAR (Single Number)")
print("=" * 50)
print(f"Value: {scalar_value}")
print(f"Type: {type(scalar_value).__name__}")
print()

```

```

import numpy as np
vector_a = np.array([1, 2, 3])
vector_b = np.array([4, 5, 6])

print("Original Vectors")
print("=" * 50)
print(f"Vector A: {vector_a}")
print(f"Vector B: {vector_b}")
print()

```

```

import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

# Sample dataset - student test scores
scores = np.array([65, 70, 75, 80, 85, 90, 95])

```

```
print("DESCRIPTIVE STATISTICS")
print("-" * 70)
print(f>Data: {scores}<div data-bbox="125 128 577 236" data-label="Text">
mean = np.mean(scores)
print("1. MEAN (Average)")
print("-" * 70)
print(f>Formula: sum(values) / count")
print(f>Calculation: ({' + '.join(map(str, scores))}) / {len(scores)}")
print(f>np.sum(scores)} / {len(scores)}")
print(f>Result: {mean}")
print(f>Interpretation: On average, the score is {mean}")
print()
```

```
median = np.median(scores)
print("2. MEDIAN (Middle Value)")
print("-" * 70)
print(f>Sorted: {np.sort(scores)}")
print(f>Middle position: {(len(scores) + 1) // 2}")
print(f>Result: {median}")
print(f>Interpretation: Half the scores are below {median}, half above")
print(f>Note: Median = Mean when data is symmetric")
print()
```

```
yield_matrix=[]

for crop_name in crops:
    row=[]
    for rainfall in rainfall_levels:
        yield_value=crop_factors[crop_name]*rainfall
        row.append(yield_value)
    yield_matrix.append(row)

print("yield prediction matrix(rows=crops,columns=rainfall levels):\n")

for i in range(len(crops)):
    print((crops[i]),":",yield_matrix[i])
```

```
std = np.std(scores)
print("3. STANDARD DEVIATION (Measure of Spread)")
print("-" * 70)
print(f>Formula:  $\sqrt{\text{sum}((x - \text{mean})^2) / n}$ ")
print(f>Result: {std:.2f}")
print(f>Interpretation:")
print(f" - Typical score varies by  $\pm\{\text{std}:.2f\}$  points from the mean")
print(f" - Small std dev = consistent scores")
print(f" - Large std dev = varied scores")
print()
```

```
variance = np.var(scores)
print("4. VARIANCE (Square of Std Dev)")
print("-" * 70)
print(f>Formula:  $\text{sum}((x - \text{mean})^2) / n$ ")
print(f>Result: {variance:.2f}")
print(f>Note: Variance = Std Dev2")
print(f" {variance:.2f} = {std:.2f}2")
print()
```

```
print("5. RANGE (Max - Min)")
print("-" * 70)
print(f>Min: {np.min(scores)}")
print(f>Max: {np.max(scores)}")
print(f>Range: {np.max(scores) - np.min(scores)}")
print()
```



```

print("6. PERCENTILES (Dividing data into parts)")
print("-" * 70)
q1 = np.percentile(scores, 25)
q2 = np.percentile(scores, 50) # Same as median
q3 = np.percentile(scores, 75)
print(f"Q1 (25th percentile): {q1} → 25% of scores below this")
print(f"Q2 (50th percentile): {q2} → 50% of scores below this (=median)")
print(f"Q3 (75th percentile): {q3} → 75% of scores below this")

```

```

file=open("/content/hhellooo.txt","w")
file.write("i am writing python code!")
file.close()

```

```

yield_prediction_str = "\n".join([str(row) for row in yield_matrix])

with open("/content/hhellooo.txt", "w") as f:
    f.write(yield_prediction_str)

print("Text file saved")

```

```

import pandas as pd

# Example: Create a simple DataFrame (replace with your actual data loading)
data = {'col1': [1, 2, 3], 'col2': [4, 5, 6]}
df = pd.DataFrame(data)

```

```

with open("yield_prediction.txt", "r") as file:
    text = file.read()

# Display the content
print(text)

```

```

sensor_temps = np.array([
    [72, 70, 71], # Day 1
    [75, 73, 74], # Day 2
    [68, 67, 69], # Day 3
    [71, 70, 72], # Day 4
    [74, 72, 73] # Day 5
])
print(sensor_temps)
print(f"Shape: {sensor_temps.shape}")

```

```

import numpy as np

def simulate_disease(days, initial_pos=(0, 0)):
    # Initialize a 4x4 grid with zeros (healthy)
    grid = np.zeros((4, 4), dtype=int)

    # Start the infection at the specified position
    grid[initial_pos] = 1

    print(f"Day 0:\n{grid}\n")

    for day in range(1, days + 1):
        # Create a copy to store the next state
        next_grid = grid.copy()

        # Iterate through each cell (Nested Loops)
        for r in range(4):
            for c in range(4):
                if grid[r, c] == 0: # If currently healthy
                    # Check neighbors: Up, Down, Left, Right
                    # We ensure indices are within [0, 3] bounds
                    neighbors = []
                    if r > 0: neighbors.append(grid[r-1, c]) # Top
                    if r < 3: neighbors.append(grid[r+1, c]) # Bottom
                    if c > 0: neighbors.append(grid[r, c-1]) # Left
                    if c < 3: neighbors.append(grid[r, c+1]) # Right

                    # If any neighbor is infected (1), this cell gets infected
                    if any(neighbors):

```

```

        next_grid[r, c] = 1

    grid = next_grid
    print(f"Day {day}:\n{grid}\n")

    # Stop early if everyone is infected
    if np.all(grid == 1):
        print("Full infection reached.")
        break

# Run simulation for 5 days starting from the top-left corner
simulate_disease(5)

```

```

Day 0:
[[1 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

```

```

Day 1:
[[1 1 0 0]
 [1 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

```

```

Day 2:
[[1 1 1 0]
 [1 1 0 0]
 [1 0 0 0]
 [0 0 0 0]]

```

```

Day 3:
[[1 1 1 1]
 [1 1 1 0]
 [1 1 0 0]
 [1 0 0 0]]

```

```

Day 4:
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 0]
 [1 1 0 0]]

```

```

Day 5:
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 1]
 [1 1 1 0]]

```

```

import numpy as np

def simulate(days, grid=None):
    if grid is None:
        grid = np.zeros((4, 4), dtype=int)
        grid[0, 0] = 1

    for day in range(days + 1):
        print(f"Day {day}:\n{grid}")
        if np.all(grid): break
        # We pad the grid with 0s to handle boundaries during shifting
        up = np.pad(grid, ((0,1),(0,0)))[1:, :]
        down = np.pad(grid, ((1,0),(0,0))[:-1, :])
        left = np.pad(grid, ((0,0),(0,1))[:, 1:])
        right = np.pad(grid, ((0,0),(1,0))[:, :-1])

        grid |= (up | down | left | right)

    simulate(3)

```

```

Day 0:
[[1 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

```

```

Day 1:
[[1 1 0 0]
 [1 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

```

```

Day 2:

```

```

[[1 1 1 0]
 [1 1 0 0]
 [1 0 0 0]
 [0 0 0 0]]
Day 3:
[[1 1 1 1]
 [1 1 1 0]
 [1 1 0 0]
 [1 0 0 0]]

```

```

import numpy as np

crops = ["Rice", "Wheat", "Maize", "Millet", "Pulses"]

yield_matrix = np.array([
    [15, 25, 40, 50, 45],
    [12, 22, 35, 42, 38],
    [10, 20, 32, 40, 36],
    [8, 15, 25, 30, 28],
    [6, 12, 20, 26, 24]
])

for i in range(5):
    print(crops[i], ":", yield_matrix[i])

```

```

Rice : [15 25 40 50 45]
Wheat : [12 22 35 42 38]
Maize : [10 20 32 40 36]
Millet : [ 8 15 25 30 28]
Pulses : [ 6 12 20 26 24]

```

```

import numpy as np

# Grid size
n = 6

# Sprinkler position (near center)
sr, sc = 2, 2

# Create 6x6 grid filled with '.'
field = np.full((n, n), '.', dtype=str)

# Apply irrigation coverage
for i in range(n):
    for j in range(n):
        distance = abs(i - sr) + abs(j - sc)
        if distance <= 2:
            field[i, j] = '*'

# Mark sprinkler location
field[sr, sc] = 'S'

# Display the grid
for row in field:
    print(" ".join(row))

```

```

. . * . . .
. * * * . .
* * S * * .
. * * * . .
. . * . . .
. . . . . .

```

```

import numpy as np

crops = ["Rice", "Wheat", "Maize", "Millet", "Pulses"]
rainfall = ["Very Low", "Low", "Moderate", "High", "Very High"]

yield_matrix = np.array([
    [15, 25, 40, 50, 45],
    [12, 22, 35, 42, 38],
    [10, 20, 32, 40, 36],
    [8, 15, 25, 30, 28],
    [6, 12, 20, 26, 24]
])

print("Rainfall:", rainfall)

```

```
for i in range(5):
    print(crops[i], ":", yield_matrix[i])
```

```
Rainfall: ['Very Low', 'Low', 'Moderate', 'High', 'Very High']
Rice : [15 25 40 50 45]
Wheat : [12 22 35 42 38]
Maize : [10 20 32 40 36]
Millet : [ 8 15 25 30 28]
Pulses : [ 6 12 20 26 24]
```

```
import numpy as np
```

```
n = 6
```

```
center_row, center_col = 2, 2
field = np.full((n, n), '.', dtype=str)
rows, cols = np.indices((n, n))
distance = np.abs(rows - center_row) + np.abs(cols - center_col)
field[distance <= 2] = '*'
```

```
field[center_row, center_col] = 'S'
```

```
for row in field:
    print(" ".join(row))
```

```
..*...
.***..
**S**
.***..
..*...
.....
```

```
import numpy as np
```

```
# Create 4x4 grid (0 = healthy, 1 = infected)
grid = np.zeros((4, 4), dtype=int)
```

```
# Initial infected cell
grid[1][1] = 1
```

```
days = 3
```

```
print("Day 0")
print(grid)
```

```
for d in range(1, days + 1):
    new_grid = grid.copy()
```

```
    for i in range(4):
        for j in range(4):
            if grid[i][j] == 1:
                if i > 0:
                    new_grid[i-1][j] = 1
                if i < 3:
                    new_grid[i+1][j] = 1
                if j > 0:
                    new_grid[i][j-1] = 1
                if j < 3:
                    new_grid[i][j+1] = 1
```

```
    grid = new_grid
```

```
    print(f"Day {d}")
    print(grid)
```

```
Day 0
[[0 0 0 0]
 [0 1 0 0]
 [0 0 0 0]
 [0 0 0 0]]
Day 1
[[0 1 0 0]
 [1 1 1 0]
 [0 1 0 0]
 [0 0 0 0]]
Day 2
```

```

[[1 1 1 0]
 [1 1 1 1]
 [1 1 1 0]
 [0 1 0 0]]
Day 3
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 1]
 [1 1 1 0]]

```

```

import numpy as np

# Create a 4x4 NumPy array (0 = healthy, 1 = infected)
field = np.array([
    [0, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0]
])

days = 3

print("Day 0")
print(field)

# Get array dimensions using shape
rows, cols = field.shape

for day in range(1, days + 1):
    # Copy array to store next state
    new_field = field.copy()

    # Nested loops to access array elements
    for i in range(rows):
        for j in range(cols):
            if field[i][j] == 1:
                # Spread to adjacent cells
                if i > 0:
                    new_field[i-1][j] = 1
                if i < rows - 1:
                    new_field[i+1][j] = 1
                if j > 0:
                    new_field[i][j-1] = 1
                if j < cols - 1:
                    new_field[i][j+1] = 1

    field = new_field

    print(f"Day {day}")
    print(field)

```

```

Day 0
[[0 0 0 0]
 [0 1 0 0]
 [0 0 0 0]
 [0 0 0 0]]
Day 1
[[0 1 0 0]
 [1 1 1 0]
 [0 1 0 0]
 [0 0 0 0]]
Day 2
[[1 1 1 0]
 [1 1 1 1]
 [1 1 1 0]
 [0 1 0 0]]
Day 3
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 1]
 [1 1 1 0]]

```

```

import numpy as np

n = 6
rows, cols = np.indices((n, n))

```

```

def print_field(field, title):
    print("\n" + title)
    print("-" * len(title))
    for row in field:
        print(" ".join(row))

field1 = np.full((n, n), '.', dtype=str)
sr, sc = 2, 2
distance = np.abs(rows - sr) + np.abs(cols - sc)
field1[distance <= 2] = '*'
field1[sr, sc] = 'S'
print_field(field1, "Scenario 1: Single Sprinkler (Diamond Pattern)")

field2 = np.full((n, n), '.', dtype=str)
radius = 2
distance = np.sqrt((rows - sr)**2 + (cols - sc)**2)
field2[distance <= radius] = '*'
field2[sr, sc] = 'S'
print_field(field2, "Scenario 2: Single Sprinkler (Circular Pattern)")

field3 = np.full((n, n), '.', dtype=str)
sr, sc = 0, 0
distance = np.abs(rows - sr) + np.abs(cols - sc)
field3[distance <= 2] = '*'
field3[sr, sc] = 'S'
print_field(field3, "Scenario 3: Corner Sprinkler")

field4 = np.full((n, n), '.', dtype=str)
sprinklers = [(1, 1), (4, 4)]
radius = 2

for sr, sc in sprinklers:
    distance = np.abs(rows - sr) + np.abs(cols - sc)
    field4[distance <= radius] = '*'
    field4[sr, sc] = 'S'

print_field(field4, "Scenario 4: Multiple Sprinklers")

field5 = np.full((n, n), '.', dtype=str)
sr, sc = 2, 2
distance = np.abs(rows - sr) + np.abs(cols - sc)
field5[distance <= 6] = '*'
field5[sr, sc] = 'S'
print_field(field5, "Scenario 5: Full Field Coverage")

```

Scenario 1: Single Sprinkler (Diamond Pattern)

```

. . * . . .
. * * * . .
* * S * * .
. * * * . .
. . * . . .
. . . . . .

```

Scenario 2: Single Sprinkler (Circular Pattern)

```

. . * . . .
. * * * . .
* * S * * .
. * * * . .
. . * . . .
. . . . . .

```

Scenario 3: Corner Sprinkler

```

S * * . . .
* * . . . .
* . . . . .
. . . . . .
. . . . . .
. . . . . .

```

Scenario 4: Multiple Sprinklers

```

* * * . . .

```

```

* S * * . .
* * * . * .
. * . * * *
. . * * S *
. . . * * *

```

Scenario 5: Full Field Coverage

```

-----
* * * * *
* * * * *
* * S * *
* * * * *
* * * * *
* * * * *

```

```

# Define two matrices
A = [
    [1, 2, 3],
    [4, 5, 6]
]

B = [
    [7, 8, 9],
    [10, 11, 12]
]

# Add the matrices
result = []
for i in range(len(A)):
    row = []
    for j in range(len(A[0])):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print(result)

```

```
[[8, 10, 12], [14, 16, 18]]
```

```

## Get matrix dimensions
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of first matrix:")
A = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"A[{i}][{j}]: ")))
    A.append(row)

print("Enter elements of second matrix:")
B = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"B[{i}][{j}]: ")))
    B.append(row)

# Add matrices
result = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print("Sum of the two matrices:")
for row in result:
    print(row)

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-1692465201.py in <cell line: 0>()
      1 ## Get matrix dimensions
----> 2 rows = int(input("Enter number of rows: "))
      3 cols = int(input("Enter number of columns: "))
      4
      5 print("Enter elements of first matrix:")

----- 1 frames -----
/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in _input_request(self, prompt, ident, parent, password)
    1217     except KeyboardInterrupt:
    1218         # re-raise KeyboardInterrupt, to truncate traceback
-> 1219         raise KeyboardInterrupt("Interrupted by user") from None
    1220     except Exception:
    1221         self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user

```

```

#Get matrix dimensions
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of first matrix:")
A = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"A[{i}][{j}]: ")))
    A.append(row)

print("Enter elements of second matrix:")
B = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"B[{i}][{j}]: ")))
    B.append(row)

# Add matrices
result = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print("Sum of the two matrices:")
for row in result:
    print(row)

```

```

# Get matrix dimensions
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of first matrix:")
A = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"A[{i}][{j}]: ")))
    A.append(row)

print("Enter elements of second matrix:")
B = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"B[{i}][{j}]: ")))
    B.append(row)

# Add matrices
result = []
for i in range(rows):

```



```

        row = []
        for j in range(cols):
            row.append(A[i][j] + B[i][j])
        result.append(row)

print("Sum of the two matrices:")
for row in result:
    print(row)

```

```

# Get matrix dimensions
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of first matrix:")
A = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"A[{i}][{j}]: ")))
    A.append(row)

print("Enter elements of second matrix:")
B = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"B[{i}][{j}]: ")))
    B.append(row)

# Add matrices
result = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print("Sum of the two matrices:")
for row in result:
    print(row)

```

```

# Get dimensions of first matrix
r1 = int(input("Enter number of rows of first matrix: "))
c1 = int(input("Enter number of columns of first matrix: "))

# Get dimensions of second matrix
r2 = int(input("Enter number of rows of second matrix: "))
c2 = int(input("Enter number of columns of second matrix: "))

# Check if multiplication is possible
if c1 != r2:
    print("Matrix multiplication not possible!")
else:
    print("Enter elements of first matrix:")
    A = []
    for i in range(r1):
        row = []
        for j in range(c1):
            row.append(int(input(f"A[{i}][{j}]: ")))
        A.append(row)

    print("Enter elements of second matrix:")
    B = []
    for i in range(r2):
        row = []
        for j in range(c2):
            row.append(int(input(f"B[{i}][{j}]: ")))
        B.append(row)

    # Initialize result matrix with zeros
    result = [[0 for _ in range(c2)] for _ in range(r1)]

    # Matrix multiplication
    for i in range(r1):

```

```

        for j in range(c2):
            for k in range(c1):
                result[i][j] += A[i][k] * B[k][j]

print("Result of matrix multiplication:")
for row in result:
    print(row)

```

```

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for val in row:
        print(val, end=" ")

```

```

print("Row-wise Traversal:")
for i in range(rows):
    for j in range(cols):
        print(matrix[i][j], end=" ")

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Row-wise Traversal:")
for i in range(rows):
    for j in range(cols):
        print(matrix[i][j], end=" ")

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Column-wise Traversal:")
for j in range(cols):
    for i in range(rows):
        print(matrix[i][j], end=" ")

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Spiral Traversal:")

top, bottom = 0, rows - 1
left, right = 0, cols - 1

while top <= bottom and left <= right:

```

```

for i in range(left, right + 1):
    print(matrix[top][i], end=" ")
top += 1

for i in range(top, bottom + 1):
    print(matrix[i][right], end=" ")
right -= 1

if top <= bottom:
    for i in range(right, left - 1, -1):
        print(matrix[bottom][i], end=" ")
    bottom -= 1

if left <= right:
    for i in range(bottom, top - 1, -1):
        print(matrix[i][left], end=" ")
    left += 1

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Zig-Zag Traversal:")
for i in range(rows):
    if i % 2 == 0:
        for j in range(cols):
            print(matrix[i][j], end=" ")
    else:
        for j in range(cols - 1, -1, -1):
            print(matrix[i][j], end=" ")

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Diagonal Traversal:")
for d in range(rows + cols - 1):
    for i in range(rows):
        j = d - i
        if 0 <= j < cols:
            print(matrix[i][j], end=" ")

```

```

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

matrix = []
print("Enter matrix elements:")
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)
print("Snake Pattern Traversal:")
for j in range(cols):
    if j % 2 == 0:
        for i in range(rows):
            print(matrix[i][j], end=" ")
    else:
        for i in range(rows - 1, -1, -1):

```

```
print(matrix[i][j], end=" ")
```

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
Tasks
```

```
x = 5
y = 2
z = x + y * 3
print(z)
```

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

```
class student:
    pass
```

```
class student:
    def __init__(self, name, roll_number, grade):
        self.name = name
        self.roll_number = roll_number
        self.grade = grade
```

```
open("/content/AI_for_Agriculture_Detailed_Story_Notebook (1).ipynb")
```

Start coding or [generate](#) with AI.

```
my_student = student("Harshitha", 83, "A")
print(f"Student Name: {my_student.name}, Roll Number: {my_student.roll_number}, Grade: {my_student.grade}")
```

```
my_second_student = student("Savitri", 26, "A")
print(f"Student Name: {my_second_student.name}, Roll Number: {my_second_student.roll_number}, Grade: {my_second_student.grade}")
```

```
all_students = []

student1 = student("Harshitha", 83, "A")
student2 = student("Savitri", 26, "A")

all_students.append(student1)
all_students.append(student2)

all_students.append(student("Shahistha", 27, "A"))

print("All student objects in the list:")
for s in all_students:
    print(f"Name: {s.name}, Roll Number: {s.roll_number}, Grade: {s.grade}")
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
```

```
np.random.seed(10)

data = pd.DataFrame({
    "Yield_Quintals": np.random.normal(5.6, 1.5, 120),
    "Water_Usage_mm": np.random.normal(220, 290, 120),
    "Cultivation_Cost_INR": np.random.normal(38000, 7000, 120),
    "Fertilizer_kg": np.random.normal(210, 35, 120),
    "Soil_pH": np.random.normal(6.5, 0.5, 120) # Added new feature: Soil pH
})

data.head()
```

Start coding or [generate](#) with AI.

/content/AI_for_Agriculture_Detailed_Story_Notebook (1).ipynb,

```
import numpy as np # Numerical engine for farm calculations

# Yield data from multiple plots (kg/hectare)
yield_data = np.array([2000, 1800, 2200, 2100, 1900]) # Each value = one plot

improvement_factor = 1.10 # 10% better practice

# Broadcasting applies the factor to all plots simultaneously
improved_yield = yield_data * improvement_factor

improved_yield # Output: improved yield for each plot
```

```
array([2200., 1980., 2420., 2310., 2090.])
```

Start coding or [generate](#) with AI.

```
import pandas as pd # Farm record management tool

# Crop-wise yield ledger
crop_yield = pd.Series(
    [3200, 2800, 3500, 3000],
    index=["Wheat", "Rice", "Maize", "Millet"]
)

crop_yield
```

```

      0
Wheat 3200
Rice  2800
Maize 3500
Millet 3000

dtype: int64
```

```
# Creating a complete farm record book
df = pd.DataFrame({
    "Crop": ["Wheat", "Rice", "Maize", "Millet"],
    "Yield_kg": [3200, 2800, 3500, 3000],
    "Water_liters": [4200, 5500, 3800, 3000],
    "Cost_INR": [18000, 20000, 17000, 15000]
})

df
```

	Crop	Yield_kg	Water_liters	Cost_INR
0	Wheat	3200	4200	18000
1	Rice	2800	5500	20000
2	Maize	3500	3800	17000
3	Millet	3000	3000	15000

```
# Selecting only high-performing crops
high_yield_df = df[df["Yield_kg"] > 3000]

high_yield_df
```

	Crop	Yield_kg	Water_liters	Cost_INR
0	Wheat	3200	4200	18000
2	Maize	3500	3800	17000

```
import numpy as np

df_missing = pd.DataFrame({
    "Rainfall_mm": [200, np.nan, 220, 210],
    "Yield_kg": [3200, 3000, np.nan, 3400]
})

# Fill missing values with mean estimation
df_filled = df_missing.fillna(df_missing.mean())

df_filled
```

	Rainfall_mm	Yield_kg
0	200.0	3200.0
1	210.0	3000.0
2	220.0	3200.0
3	210.0	3400.0

```
df_group = pd.DataFrame({
    "Crop": ["Wheat", "Rice", "Wheat", "Maize", "Rice"],
    "Yield_kg": [3000, 2800, 3400, 3600, 2900]
})

df_group.groupby("Crop").mean()
```

	Yield_kg
Crop	
Maize	3600.0
Rice	2850.0
Wheat	3200.0

```
from sklearn.linear_model import LinearRegression

# Historical data (past seasons)
rainfall = np.array([[200], [180], [220], [210]])
yield_data = np.array([3200, 3000, 3500, 3400])

model = LinearRegression()
model.fit(rainfall, yield_data) # Learning from past experience
```

```
# Predict for upcoming season
predicted_yield = model.predict([[205]])

predicted_yield
```

```
array([3307.14285714])
```

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans

rainfall = np.array([[200],[180],[220],[210]])
yield_kg = np.array([3200,3000,3500,3400])
sup = LinearRegression()
sup.fit(rainfall, yield_kg)
sup.predict([[205]])

soil = np.array([[6.5,30],[7.2,45],[6.6,32],[7.5,50]])
KMeans(n_clusters=2, random_state=0).fit_predict(soil)
```

```
array([0, 1, 0, 1], dtype=int32)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.DataFrame({
    "Rainfall": [200,180,220,210,190,230],
    "Yield": [3200,3000,3500,3400,3100,3600]
})

X = df[["Rainfall"]]
y = df[["Yield"]]
train_test_split(X, y, test_size=0.3, random_state=42)
```

```
[  Rainfall
5      230
2      220
4      190
3      210,
   Rainfall
0      200
1      180,
5     3600
2     3500
4     3100
3     3400
Name: Yield, dtype: int64,
0     3200
1     3000
Name: Yield, dtype: int64]
```

```
import pandas as pd
data = {
    "Rainfall_mm": [200, 180, 220, 210, 190],
    "Soil_Moisture": [30, 28, 35, 33, 29],
    "Yield_kg": [3200, 3000, 3500, 3400, 3100]
}
df = pd.DataFrame(data)
df
```

	Rainfall_mm	Soil_Moisture	Yield_kg
0	200	30	3200
1	180	28	3000
2	220	35	3500
3	210	33	3400
4	190	29	3100

```
from sklearn.linear_model import LinearRegression
X = df[["Rainfall_mm", "Soil_Moisture"]]
```

```
y = df["Yield_kg"]
model = LinearRegression()
model.fit(X, y)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
predicted_yield = model.predict(X)
predicted_yield
```

```
array([3202.5, 3002.5, 3515. , 3377.5, 3102.5])
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = {
    "Rainfall": [200, 300, 400, 500, 600, 700, 800],
    "Fertilizer": [50, 60, 70, 80, 90, 100, 110],
    "Temperature": [20, 22, 24, 26, 28, 30, 32],
    "Yield": [1.2, 1.8, 2.6, 3.1, 3.7, 4.2, 4.8]
}

df = pd.DataFrame(data)
df
```

	Rainfall	Fertilizer	Temperature	Yield
0	200	50	20	1.2
1	300	60	22	1.8
2	400	70	24	2.6
3	500	80	26	3.1
4	600	90	28	3.7
5	700	100	30	4.2
6	800	110	32	4.8

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = {
    "Rainfall": [200, 300, 400, 500, 600, 700, 800],
    "Fertilizer": [50, 60, 70, 80, 90, 100, 110],
    "Temperature": [20, 22, 24, 26, 28, 30, 32],
    "Yield": [1.2, 1.8, 2.6, 3.1, 3.7, 4.2, 4.8]
}

df = pd.DataFrame(data)
df

X = df[["Rainfall", "Fertilizer", "Temperature"]]
y = df["Yield"]

model = LinearRegression()
model.fit(X, y)

new_data = pd.DataFrame({
    "Rainfall": [550],
    "Fertilizer": [85],
    "Temperature": [27]
})

predicted_yield = round(model.predict(new_data)[0], 2)

print("\n" + "="*40)
print(" CROP YIELD PREDICTION ")
print("="*40)
print(f" Rainfall      : {new_data['Rainfall'][0]} mm")
```



```
print(f" Fertilizer  : {new_data['Fertilizer'][0]} kg/ha")
print(f" Temperature : {new_data['Temperature'][0]} °C")
print("-"*40)
print(f" Predicted Yield : {predicted_yield} tons/hectare")
print("-"*40)
```

```
=====
CROP YIELD PREDICTION
=====
Rainfall   : 550 mm
Fertilizer  : 85 kg/ha
Temperature : 27 °C
-----
Predicted Yield : 3.36 tons/hectare
=====
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([12000, 15000, 18000, 21000, 24000]).reshape(-1, 1)

y = np.array([8000, 11000, 13500, 16000, 19000])
```

```
model = LinearRegression()
model.fit(X, y)
```

```
▼ LinearRegression ⓘ ?
LinearRegression()
```

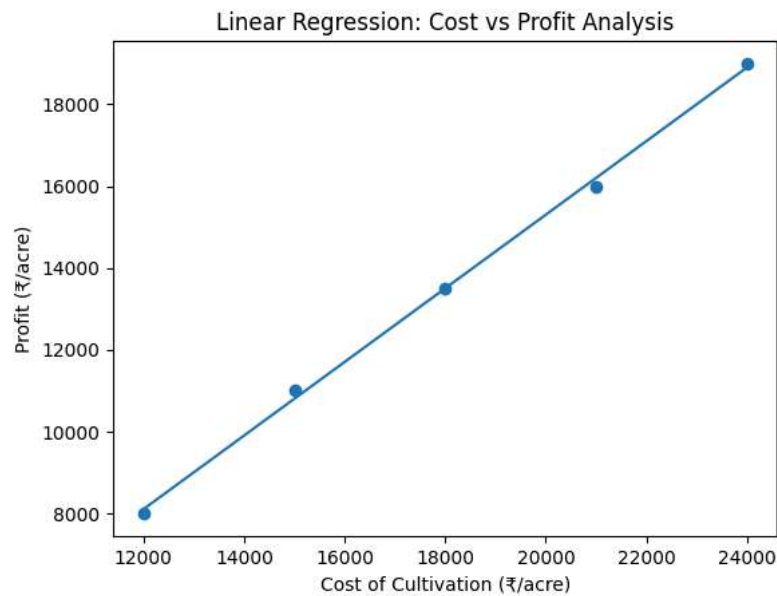
```
predicted_profit = model.predict([[20000]])
print("Predicted Profit (₹/acre):", int(predicted_profit[0]))
```

```
Predicted Profit (₹/acre): 15300
```

```
print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)
```

```
Slope (m): 0.9
Intercept (c): -2700.0
```

```
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Cost of Cultivation (₹/acre)")
plt.ylabel("Profit (₹/acre)")
plt.title("Linear Regression: Cost vs Profit Analysis")
plt.show()
```



```
import pandas as pd

data = {
    "Yield_kg": [3200, 3000, 3500, 2800, 3600, 2900],
    "Soil_Moisture": [30, 28, 35, 25, 36, 27]
}

df = pd.DataFrame(data)
df
```

	Yield_kg	Soil_Moisture
0	3200	30
1	3000	28
2	3500	35
3	2800	25
4	3600	36
5	2900	27

```
import numpy as np

plots = np.array([30, 28, 35, 25, 36, 27])
center = 30

plots - center

array([ 0, -2,  5, -5,  6, -3])
```

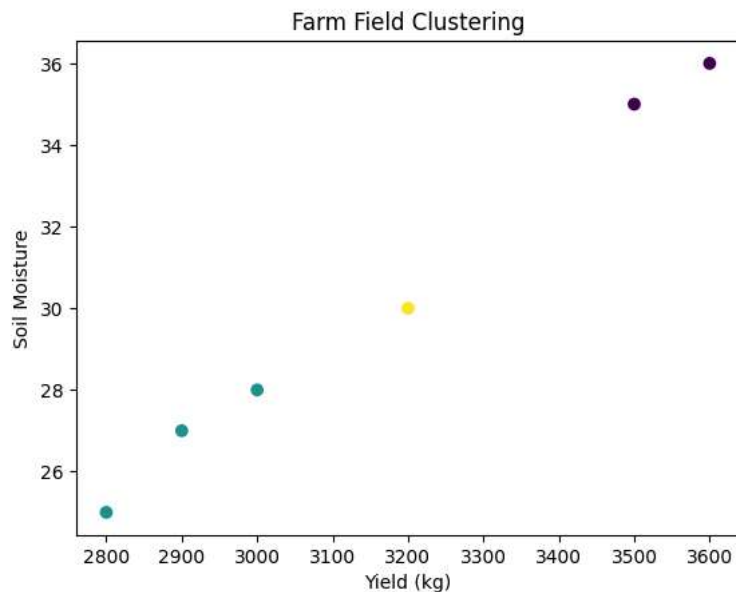
```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
df["Zone"] = kmeans.fit_predict(df)
df
```

	Yield_kg	Soil_Moisture	Zone
0	3200	30	2
1	3000	28	1
2	3500	35	0
3	2800	25	1
4	3600	36	0
5	2900	27	1

```
import matplotlib.pyplot as plt

plt.scatter(df["Yield_kg"], df["Soil_Moisture"], c=df["Zone"])
plt.xlabel("Yield (kg)")
plt.ylabel("Soil Moisture")
plt.title("Farm Field Clustering")
plt.show()
```



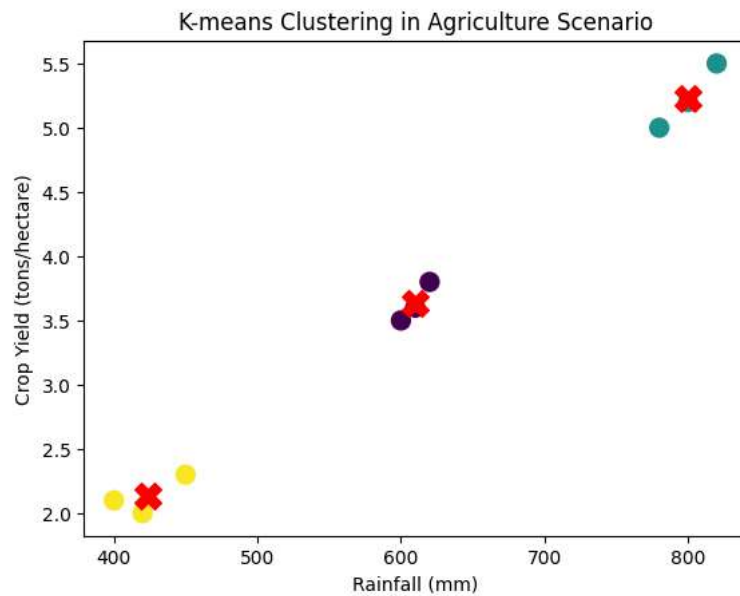
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Columns: [Rainfall (mm), Crop Yield (tons/hectare)]
data = np.array([
    [400, 2.1], [450, 2.3], [420, 2.0],
    [600, 3.5], [620, 3.8], [610, 3.6],
    [800, 5.2], [820, 5.5], [780, 5.0]
])

# Apply K-means
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(data)
labels = kmeans.labels_

# Plot clusters
plt.scatter(data[:,0], data[:,1], c=labels, cmap='viridis', s=100)
plt.scatter(
    kmeans.cluster_centers_[0,0],
    kmeans.cluster_centers_[0,1],
    color='red', marker='X', s=200
)

plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (tons/hectare)")
plt.title("K-means Clustering in Agriculture Scenario")
plt.show()
```



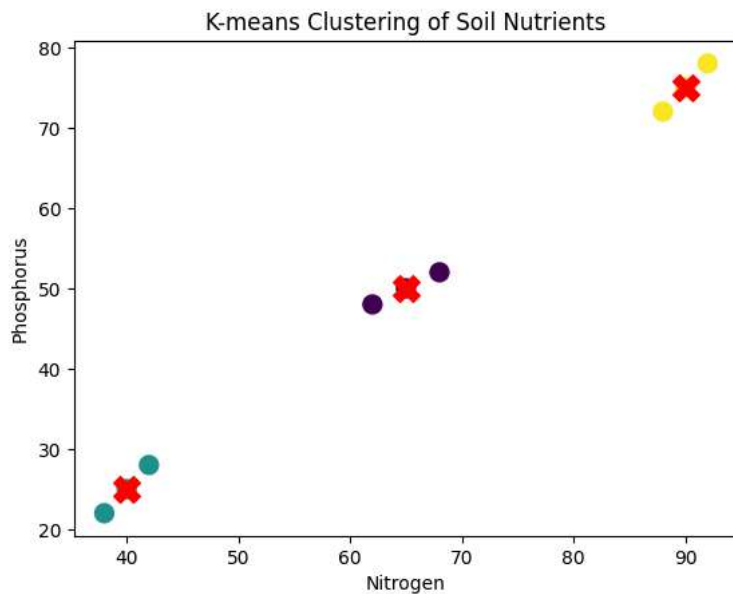
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Soil nutrients data: [Nitrogen, Phosphorus, Potassium]
soil_data = np.array([
    [40, 25, 30],
    [42, 28, 35],
    [38, 22, 28],
    [65, 50, 55],
    [68, 52, 58],
    [62, 48, 53],
    [90, 75, 80],
    [92, 78, 85],
    [88, 72, 78]
])

# Apply K-means
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(soil_data)
labels = kmeans.labels_

# Plot (N vs P)
plt.scatter(soil_data[:,0], soil_data[:,1], c=labels, cmap='viridis', s=100)
plt.scatter(
    kmeans.cluster_centers_[0,0],
    kmeans.cluster_centers_[0,1],
    color='red', marker='X', s=200
)

plt.xlabel("Nitrogen")
plt.ylabel("Phosphorus")
plt.title("K-means Clustering of Soil Nutrients")
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Data: [N, P, K]
X = np.array([
    [40, 25, 30],
    [42, 28, 35],
    [38, 22, 28],
    [65, 50, 55],
    [68, 52, 58],
    [62, 48, 53],
    [90, 75, 80],
    [92, 78, 85],
    [88, 72, 78]
])

X = StandardScaler().fit_transform(X)

k = 3

# Reference clustering
ref = KMeans(n_clusters=k, random_state=0, n_init=10).fit_predict(X)

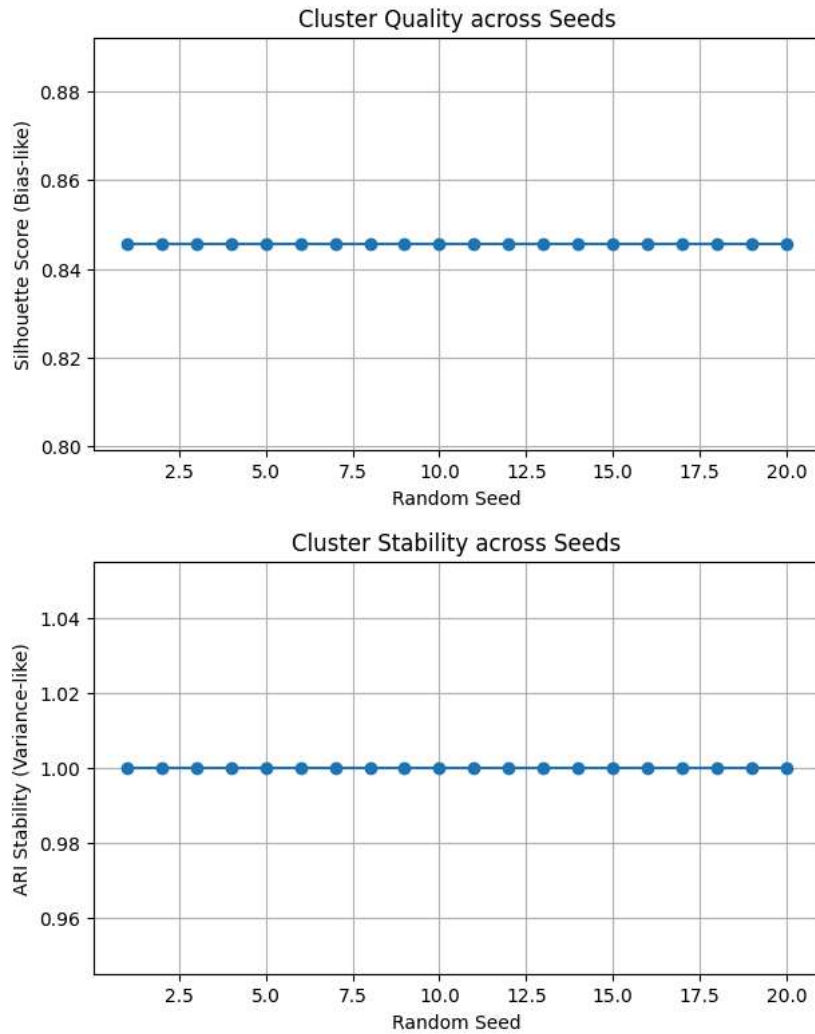
sil_scores = []
ari_scores = []
seeds = range(1, 21)

for seed in seeds:
    labels = KMeans(n_clusters=k, random_state=seed, n_init=10).fit_predict(X)
    sil_scores.append(silhouette_score(X, labels))      # Bias-like
    ari_scores.append(adjusted_rand_score(ref, labels)) # Variance-like

# ----- Graphs -----
plt.figure(figsize=(7,4))
plt.plot(list(seeds), sil_scores, marker="o")
plt.xlabel("Random Seed")
plt.ylabel("Silhouette Score (Bias-like)")
plt.title("Cluster Quality across Seeds")
plt.grid(True)
plt.show()

plt.figure(figsize=(7,4))
plt.plot(list(seeds), ari_scores, marker="o")
plt.xlabel("Random Seed")
plt.ylabel("ARI Stability (Variance-like)")
plt.title("Cluster Stability across Seeds")
plt.grid(True)
plt.show()
```

```
print("Average Silhouette =", round(np.mean(sil_scores), 3))
print("Average ARI =", round(np.mean(ari_scores), 3))
```



Average Silhouette = 0.846
Average ARI = 1.0

```
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = iris.target_names[y]

print(df.head())
print(f"\nShape: {df.shape}")
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	species
0	setosa
1	setosa
2	setosa
3	setosa

```
4 setosa
```

```
Shape: (150, 5)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0, gamma='scale')
```

```
from sklearn.model_selection import GridSearchCV

params = {'C':[0.1,1,10,100], 'gamma':[0.01,0.1,1]}
grid = GridSearchCV(SVC(kernel='rbf'), params, cv=5)
grid.fit(X_scaled, y)
best_model = grid.best_estimator_
```

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(SVC(kernel='rbf'), X_scaled, y, cv=5)
```

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load dataset
cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=cancer.feature_names)
df['diagnosis'] = cancer.target_names[y]

print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness \
0         17.99         10.38         122.80      1001.0         0.11840
1         20.57         17.77         132.90      1326.0         0.08474
2         19.69         21.25         130.00      1203.0         0.10960
3         11.42         20.38          77.58       386.1         0.14250
4         20.29         14.34         135.10      1297.0         0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry \
0         0.27760         0.3001         0.14710         0.2419
1         0.07864         0.0869         0.07017         0.1812
2         0.15990         0.1974         0.12790         0.2069
3         0.28390         0.2414         0.10520         0.2597
4         0.13280         0.1980         0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area \
0         0.07871  ...         17.33         184.60         2019.0
1         0.05667  ...         23.41         158.80         1956.0
2         0.05999  ...         25.53         152.50         1709.0
3         0.09744  ...         26.50          98.87         567.7
4         0.05883  ...         16.67         152.20         1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points \
0         0.1622         0.6656         0.7119         0.2654
1         0.1238         0.1866         0.2416         0.1860
2         0.1444         0.4245         0.4504         0.2430
3         0.2098         0.8663         0.6869         0.2575
4         0.1374         0.2050         0.4000         0.1625

   worst symmetry  worst fractal dimension  diagnosis
0         0.4601         0.11890  malignant
1         0.2750         0.08902  malignant
2         0.3613         0.08758  malignant
3         0.6638         0.17300  malignant
4         0.2364         0.07678  malignant
```

```
[5 rows x 31 columns]
```

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)

model = KNeighborsClassifier()
model.fit(X_train, y_train)

print(model.score(X_test, y_test))

```

0.956140350877193

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

from sklearn.model_selection import GridSearchCV

params = {'n_neighbors': range(1, 21)}
grid = GridSearchCV(KNeighborsClassifier(), params, cv=5)
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)

```

{'n_neighbors': 8}
0.9670329670329672

Start coding or [generate](#) with AI.

```

from sklearn.datasets import load_wine
import pandas as pd

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=wine.feature_names)
df['cultivar'] = wine.target_names[y]

print(df.describe())

```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium \
count	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573
std	0.811827	1.117146	0.274344	3.339564	14.282484
min	11.030000	0.740000	1.360000	10.600000	70.000000
25%	12.362500	1.602500	2.210000	17.200000	88.000000
50%	13.050000	1.865000	2.360000	19.500000	98.000000
75%	13.677500	3.082500	2.557500	21.500000	107.000000
max	14.830000	5.800000	3.230000	30.000000	162.000000

	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins \
count	178.000000	178.000000	178.000000	178.000000
mean	2.295112	2.029270	0.361854	1.590899
std	0.625851	0.998859	0.124453	0.572359
min	0.980000	0.340000	0.130000	0.410000
25%	1.742500	1.205000	0.270000	1.250000
50%	2.355000	2.135000	0.340000	1.555000
75%	2.800000	2.875000	0.437500	1.950000
max	3.880000	5.080000	0.660000	3.580000

	color_intensity	hue	od280/od315_of_diluted_wines	proline
count	178.000000	178.000000	178.000000	178.000000
mean	5.058090	0.957449	2.611685	746.893258
std	2.318286	0.228572	0.709990	314.907474
min	1.280000	0.480000	1.270000	278.000000

25%	3.220000	0.782500	1.937500	500.500000
50%	4.690000	0.965000	2.780000	673.500000
75%	6.200000	1.120000	3.170000	985.000000
max	13.000000	1.710000	4.000000	1680.000000

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
wine = load_wine()
```

```
X = wine.data
y = wine.target
```

```
print("Feature Names:", wine.feature_names)
print("Classes:", wine.target_names)
```

```
Feature Names: ['alcohol', 'malic_acid', 'ash', 'alkalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_p
Classes: ['class_0' 'class_1' 'class_2']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
model = SVC()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

```
model = SVC(kernel='rbf')
```

```
from sklearn.model_selection import GridSearchCV
```

```
params = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.01, 0.1, 1],
    'kernel': ['rbf']
}
```

```
grid = GridSearchCV(SVC(), params, cv=5)
grid.fit(X_train, y_train)
```

```
print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
```

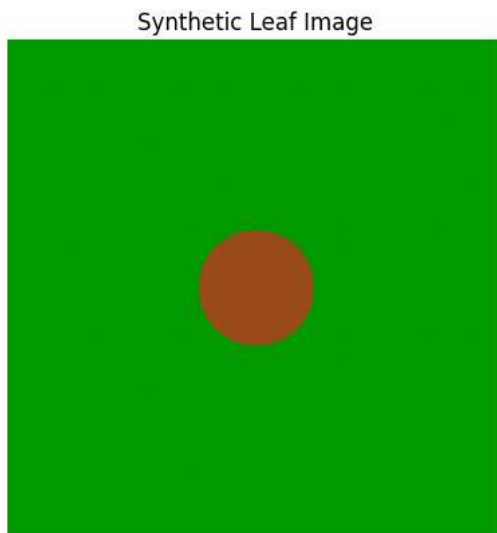
```
Best Parameters: {'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}
Best CV Accuracy: 0.9788177339901478
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
img = np.zeros((256,256,3))
img[:, :, 1] = 0.6

for i in range(256):
    for j in range(256):
        if (i-128)**2 + (j-128)**2 < 30**2:
            img[i,j] = [0.6,0.3,0.1]

plt.figure()
plt.imshow(img)
plt.title('Synthetic Leaf Image')
plt.axis('off')
plt.show()
```



```
img_input = np.expand_dims(img,axis=0)
model = unet_model() # Instantiate the model
prediction = model.predict(img_input)

plt.figure()
plt.imshow(prediction[0,:,:,:])
plt.title('Predicted Mask (Random Weights)')
plt.axis('off')
plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-1233951874.py in <cell line: 0>()
      1 img_input = np.expand_dims(img,axis=0)
----> 2 model = unet_model() # Instantiate the model
      3 prediction = model.predict(img_input)
      4
      5 plt.figure()

/tmp/ipython-input-233501306.py in unet_model()
      1 def unet_model():
----> 2     inputs = layers.Input((256,256,3))
      3
      4     c1 = layers.Conv2D(32,(3,3),activation='relu',padding='same')(inputs)
      5     c1 = layers.Conv2D(32,(3,3),activation='relu',padding='same')(c1)

NameError: name 'layers' is not defined
```

```
layer_outputs = [layer.output for layer in model.layers if isinstance(layer, tf.keras.layers.Conv2D)]
feature_model = models.Model(inputs=model.input, outputs=layer_outputs)

features = feature_model.predict(img_input)
```

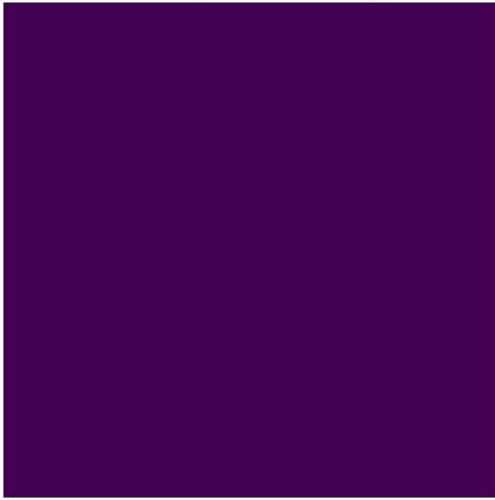
```

for i in range(min(4,len(features))):
    plt.figure()
    plt.imshow(features[i][0,:,:,:])
    plt.title(f'Feature Map from Conv Layer {i+1}')
    plt.axis('off')
    plt.show()

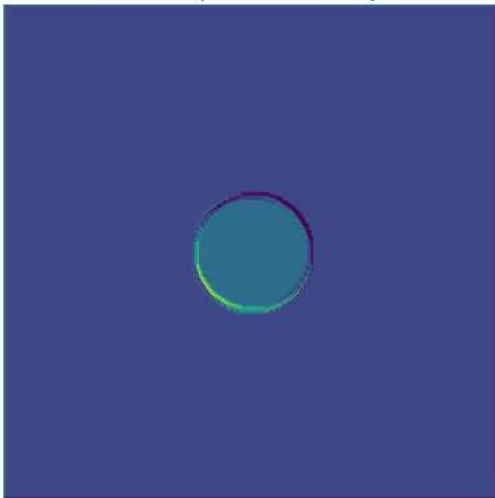
```

1/1 — 0s 438ms/step

Feature Map from Conv Layer 1



Feature Map from Conv Layer 2



Feature Map from Conv Layer 3



```

from sklearn import svm
from sklearn.datasets import make_classification
import numpy as np

X, y = make_classification(n_samples=100, n_features=2,
                          n_redundant=0, n_informative=2,
                          random_state=42)

clf = svm.SVC(
    kernel='poly',
    C=1.00,
    gamma=0.10,
    degree=3,
    random_state=42
)

clf.fit(X, y)

```

```

support_vectors = clf.support_vectors_
print(f"Number of support vectors: {len(support_vectors)}")

predictions = clf.predict(X)

decision_values = clf.decision_function(X)
print(f"Decision values range: [{decision_values.min():.2f}, {decision_values.max():.2f}]")

```

```

Number of support vectors: 83
Decision values range: [-3.39, 4.56]

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create a sample dataset for demonstration since 'plant_disease.csv' was not found previously
data = pd.DataFrame({
    'feature1': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'feature2': [10, 9, 8, 7, 6, 5, 4, 3, 2, 1],
    'feature3': [100, 101, 102, 103, 104, 105, 106, 107, 108, 109],
    'disease': ['Rust', 'Mildew', 'Rust', 'Blight', 'Mildew', 'Rust', 'Blight', 'Mildew', 'Rust', 'Blight']
})

# Encode categorical target variable (Disease names → numbers)
label_encoder = LabelEncoder()
data['disease'] = label_encoder.fit_transform(data['disease'])

# Separate features and target
X = data.drop('disease', axis=1)
y = data['disease']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Visualize Decision Tree
plt.figure(figsize=(15,10))
plot_tree(model,
          feature_names=X.columns,
          class_names=label_encoder.classes_,
          filled=True)
plt.title("Decision Tree for Plant Disease Diagnosis")
plt.show()
df

```

Accuracy: 0.0

Confusion Matrix:

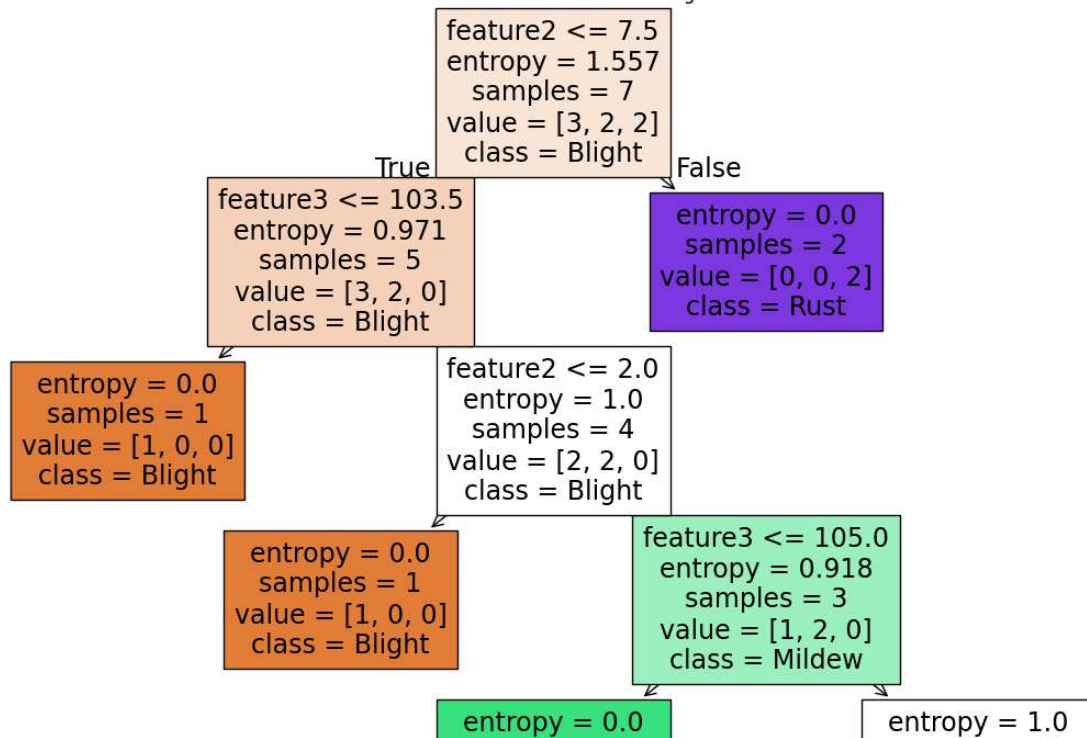
```
[[0 0 0]
 [0 0 1]
 [1 1 0]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	0.0
1	0.00	0.00	0.00	1.0
2	0.00	0.00	0.00	2.0
accuracy			0.00	3.0
macro avg	0.00	0.00	0.00	3.0
weighted avg	0.00	0.00	0.00	3.0

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Decision Tree for Plant Disease Diagnosis



```
import numpy as np
```

```
# Input values (could be from convolution layer)
```

```
x = np.array([-2, -1, 0, 1, 2, 3])
```

```
# Tanh activation
```

```
output = np.tanh(x)
```

```
# Result: [-0.964, -0.762, 0, 0.762, 0.964, 0.995]
```

```
# All values between -1 and 1
```

```
print(output)
```

```
3      900      24      50      120      35
[-0.96402758 -0.76159416  0.      0.76159416  0.96402758  0.99505475]
4      650      29      38      85      24
```

```
import numpy as np
```

```
# Input values (could be from convolution layer)
```

```
x = np.array([-2, -1, 0, 1, 2, 3])
```

```
# Sigmoid activation
```

```
output = 1 / (1 + np.exp(-x))
```

```
# Result: [0.119, 0.269, 0.5, 0.731, 0.881, 0.953]
```

```
# All values between 0 and 1
```

```
print(output)
```

```
[0.11920292 0.26894142 0.5          0.73105858 0.88079708 0.95257413]
```

```
import numpy as np
```

```
# Input image
```

```
image = np.array([
    [1, 2, 0, 0, 1],
    [3, 1, 2, 1, 0],
    [0, 1, 3, 1, 2],
    [2, 0, 1, 2, 1],
    [1, 2, 1, 0, 0]
])
```

```
# Edge detection kernel
```

```
kernel = np.array([
    [1, 0, -1],
    [1, 0, -1],
    [1, 0, -1]
])
```

```
# Convolution operation
```

```
output = []
for i in range(image.shape[0] - kernel.shape[0] + 1):
    row = []
    for j in range(image.shape[1] - kernel.shape[1] + 1):
        # Element-wise multiplication and sum
        patch = image[i:i+3, j:j+3]
        value = np.sum(patch * kernel)
        row.append(value)
    output.append(row)
```

```
print(output)
```

```
# Result: Feature map highlighting vertical edges
```

```
[[np.int64(-1), np.int64(2), np.int64(2)], [np.int64(-1), np.int64(-2), np.int64(3)], [np.int64(-2), np.int64(0), np.int64(2)]]
```

```
import numpy as np
```

```
leaf = np.array([
    [2, 3, 2, 1, 0],
    [3, 5, 4, 2, 1],
    [2, 4, 9, 4, 2],
    [1, 2, 4, 5, 3],
    [0, 1, 2, 3, 2]
])
```

```
spot_kernel = np.array([
```

```
    [0, 1, 0],
    [1,-4, 1],
    [0, 1, 0]
])
```

```
blight_kernel = np.array([
```

```
    [1, 0,-1],
    [1, 0,-1],
    [1, 0,-1]
])
```

```
rust_kernel = np.array([
```

```
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])
```

```
def apply_filter(image, kernel):
```

```
    output = []
    for i in range(image.shape[0] - kernel.shape[0] + 1):
        row = []
        for j in range(image.shape[1] - kernel.shape[1] + 1):
            patch = image[i:i+3, j:j+3]
            value = np.sum(patch * kernel)
            row.append(value)
```

```

        output.append(row)
    return np.array(output)

spot_result = apply_filter(leaf, spot_kernel)
blight_result = apply_filter(leaf, blight_kernel)
rust_result = apply_filter(leaf, rust_kernel)

print("Spot Disease Detection:\n", spot_result)
print("\nBlight Detection:\n", blight_result)
print("\nRust Detection:\n", rust_result)

```

Spot Disease Detection:

```

[[ -6  2  2]
 [  2 -20  2]
 [  2  2 -6]]

```

Blight Detection:

```

[[ -8  5 12]
 [-11  0 11]
 [-12 -5  8]]

```

Rust Detection:

```

[[11  2 -7]
 [ 2 42  2]
 [-7  2 11]]

```

```

import numpy as np

# Simulated leaf image (pixel intensity values)
image = np.array([
    [34, 67, 23, 12, 90],
    [88,120, 56, 34, 21],
    [45, 78,200, 67, 32],
    [12, 56, 89,145, 76],
    [ 9, 23, 67, 98, 54]
])

# Disease detection kernel (example: rust pattern detector)
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

# Convolution
output = []

for i in range(image.shape[0] - 2):
    row = []
    for j in range(image.shape[1] - 2):
        patch = image[i:i+3, j:j+3]
        value = np.sum(patch * kernel)
        row.append(value)
    output.append(row)

output = np.array(output)

# Normalize values to pixel range (0-255)
output = output - output.min()
output = (output / output.max()) * 255
output = output.astype(np.uint8)

# Print pixel output
print("Output Image Pixel Values:\n")
for r in output:
    for p in r:
        print(f"{p:3}", end=" ")
    print()

```

Output Image Pixel Values:

```

128 16  0
40 255 24
33 44 152

```

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

```

```

# Step 1: Create Synthetic Potato Leaf Image
# Green background (leaf)
leaf = np.zeros((400, 400, 3), dtype=np.uint8)
leaf[:] = (0, 180, 0) # Green color in BGR

# Add synthetic brown spots (disease)
cv2.circle(leaf, (120, 150), 30, (42, 42, 165), -1) # Spot 1
cv2.circle(leaf, (250, 220), 40, (19, 69, 139), -1) # Spot 2
cv2.circle(leaf, (300, 100), 25, (60, 60, 150), -1) # Spot 3

# Step 2: Convert to HSV
hsv = cv2.cvtColor(leaf, cv2.COLOR_BGR2HSV)

# Step 3: Define brown color range (disease spots)
lower_brown = np.array([5, 100, 20])
upper_brown = np.array([25, 255, 200])

mask = cv2.inRange(hsv, lower_brown, upper_brown)

# Step 4: Morphological operations to clean noise
kernel = np.ones((5,5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

# Step 5: Find contours
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Draw contours
output = leaf.copy()
cv2.drawContours(output, contours, -1, (0,0,255), 2)

# Step 6: Calculate infected area percentage
total_area = leaf.shape[0] * leaf.shape[1]
infected_area = cv2.countNonZero(mask)
disease_percentage = (infected_area / total_area) * 100

print("Number of disease spots detected:", len(contours))
print("Disease affected area: {:.2f}%".format(disease_percentage))

# Show results
plt.figure(figsize=(10,5))

plt.subplot(1,3,1)
plt.title("Synthetic Leaf")
plt.imshow(cv2.cvtColor(leaf, cv2.COLOR_BGR2RGB))
plt.axis("off")

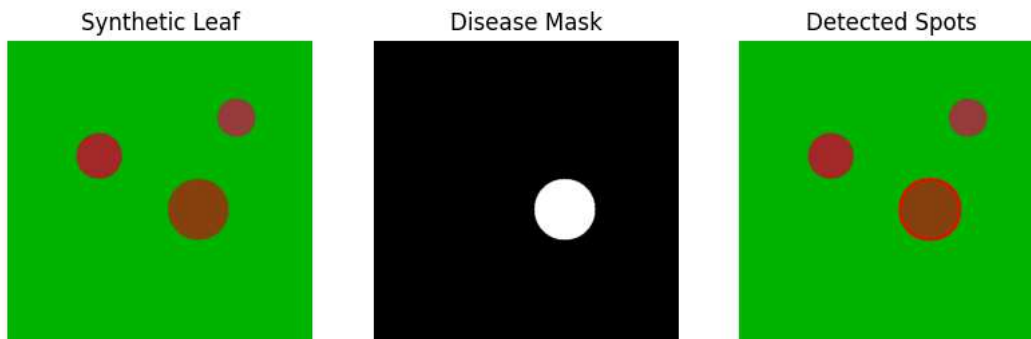
plt.subplot(1,3,2)
plt.title("Disease Mask")
plt.imshow(mask, cmap='gray')
plt.axis("off")

plt.subplot(1,3,3)
plt.title("Detected Spots")
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()

```

Number of disease spots detected: 1
Disease affected area: 3.14%




```

import numpy as np
import matplotlib.pyplot as plt

# Input image (leaf pixels)
image = np.array([
    [34, 67, 23, 12, 90],
    [88,120, 56, 34, 21],
    [45, 78,200, 67, 32],
    [12, 56, 89,145, 76],
    [ 9, 23, 67, 98, 54]
])

# Disease detection kernel
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

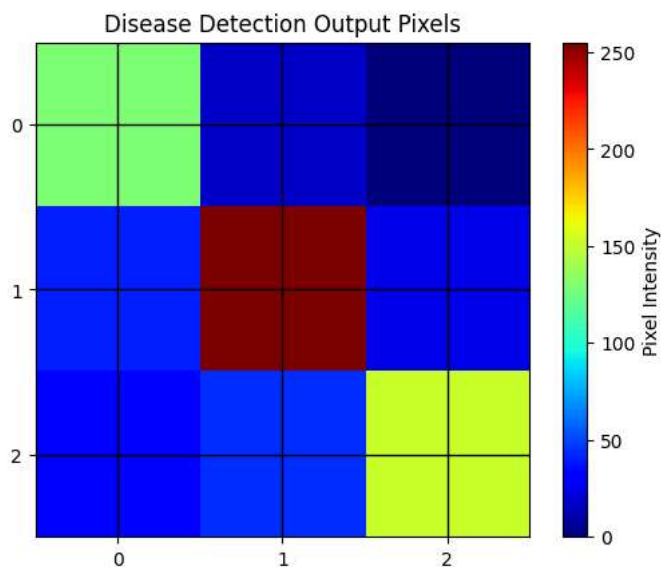
# Convolution
output = []
for i in range(image.shape[0] - 2):
    row = []
    for j in range(image.shape[1] - 2):
        patch = image[i:i+3, j:j+3]
        value = np.sum(patch * kernel)
        row.append(value)
    output.append(row)

output = np.array(output)

# Normalize to 0-255
output = output - output.min()
output = (output / output.max()) * 255

# ----- Display as Colored Pixels -----
plt.imshow(output, cmap='jet') # change colormap if needed
plt.colorbar(label="Pixel Intensity")
plt.title("Disease Detection Output Pixels")
plt.xticks(range(output.shape[1]))
plt.yticks(range(output.shape[0]))
plt.grid(color='black', linestyle='-', linewidth=1)
plt.show()

```



```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Disease Leaf Image -----
# Replace filename with your image name
image = cv2.imread("disease_leaf.jpg")

```

```

# Convert BGR → Grayscale
gray = cv2.cvtColor(i)

# ----- Disease Detection Kernel -----
# Texture highlight kernel (good for spots/rust patterns)
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

# ----- Convolution -----
h, w = gray.shape
output = np.zeros((h-2, w-2))

for i in range(h-2):
    for j in range(w-2):
        patch = gray[i:i+3, j:j+3]
        output[i, j] = np.sum(patch * kernel)

# ----- Normalize Pixels 0-255 -----
output = output - output.min()
output = (output / output.max()) * 255
output = output.astype(np.uint8)

# ----- Display Colored Pixel Boxes -----
plt.imshow(output, cmap='jet')
plt.title("Detected Disease Pixels")
plt.colorbar(label="Intensity")
plt.grid(color='black', linestyle='-', linewidth=0.3)
plt.show()

```

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Disease Leaf Image -----
# Replace filename with your image name
image = cv2.imread("/content/WhatsApp Image 2026-02-23 at 2.37.52 PM (1).jpeg")

# Convert BGR → Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# ----- Disease Detection Kernel -----
# Texture highlight kernel (good for spots/rust patterns)
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

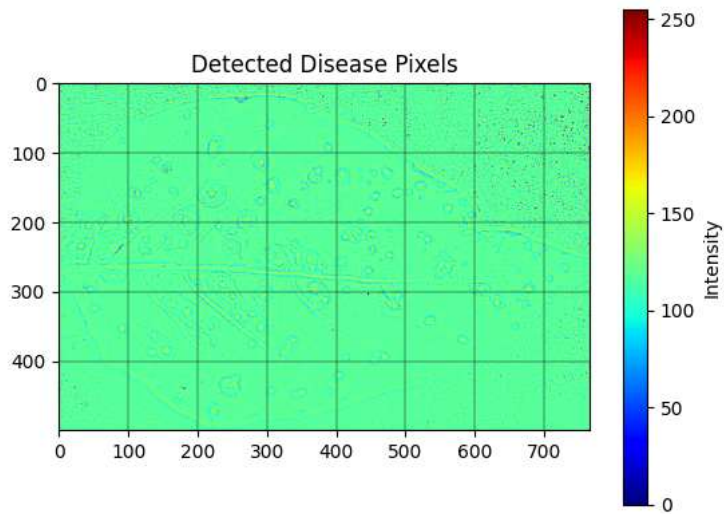
# ----- Convolution -----
h, w = gray.shape
output = np.zeros((h-2, w-2))

for i in range(h-2):
    for j in range(w-2):
        patch = gray[i:i+3, j:j+3]
        output[i, j] = np.sum(patch * kernel)

# ----- Normalize Pixels 0-255 -----
output = output - output.min()
output = (output / output.max()) * 255
output = output.astype(np.uint8)

# ----- Display Colored Pixel Boxes -----
plt.imshow(output, cmap='jet')
plt.title("Detected Disease Pixels")
plt.colorbar(label="Intensity")
plt.grid(color='black', linestyle='-', linewidth=0.3)
plt.show()

```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Disease Leaf Image -----
image = cv2.imread("/content/WhatsApp Image 2026-02-23 at 2.37.52 PM (1).jpeg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# ----- Disease Detection Kernel -----
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

# ----- Convolution -----
h, w = gray.shape
output = np.zeros((h-2, w-2))

for i in range(h-2):
    for j in range(w-2):
        patch = gray[i:i+3, j:j+3]
        value = np.sum(patch * kernel)

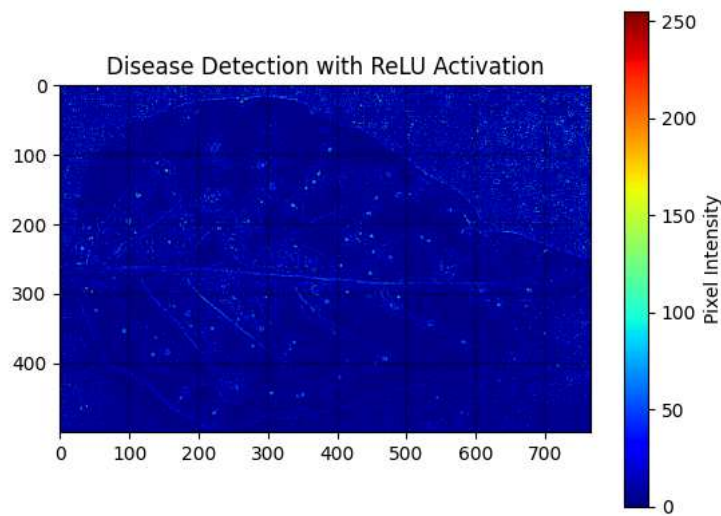
        # ----- ReLU Activation -----
        value = max(0, value)

        output[i, j] = value

# ----- Normalize Pixels -----
if output.max() != 0:
    output = (output / output.max()) * 255

output = output.astype(np.uint8)

# ----- Display Result -----
plt.imshow(output, cmap='jet')
plt.title("Disease Detection with ReLU Activation")
plt.colorbar(label="Pixel Intensity")
plt.grid(color='black', linestyle='-', linewidth=0.3)
plt.show()
```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image = cv2.imread("/content/WhatsApp Image 2026-02-23 at 2.37.52 PM (1).jpeg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# ----- Kernel -----
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

# ----- Convolution + ReLU -----
h, w = gray.shape
output = np.zeros((h-2, w-2))

for i in range(h-2):
    for j in range(w-2):
        patch = gray[i:i+3, j:j+3]
        value = np.sum(patch * kernel)
        value = max(0, value) # ReLU
        output[i, j] = value

# Normalize
if output.max() != 0:
    output = (output/output.max())*255

output = output.astype(np.uint8)

# ----- Color Interpretation -----
colored = np.zeros((output.shape[0], output.shape[1], 3), dtype=np.uint8)

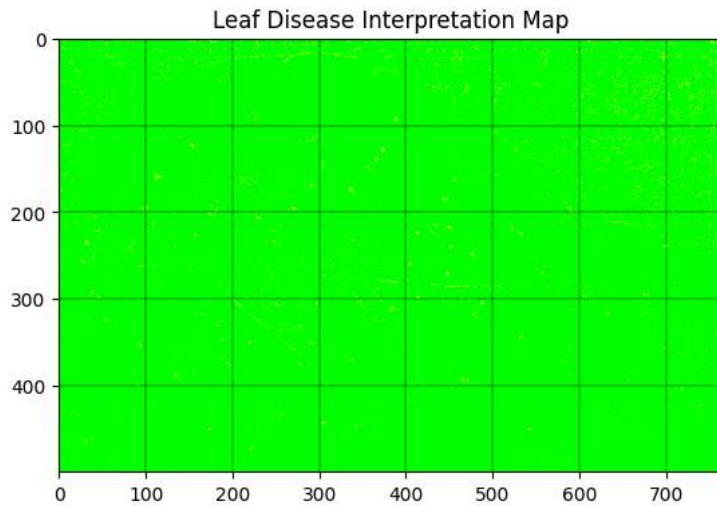
for i in range(output.shape[0]):
    for j in range(output.shape[1]):
        pixel = output[i, j]

        if pixel < 60:                # Healthy
            colored[i,j] = [0,255,0]    # Green

        elif pixel < 150:             # Spotted area
            colored[i,j] = [255,255,0]    # Yellow

        else:                         # Severe spot centre
            colored[i,j] = [180,180,180]  # Grey

# ----- Display -----
plt.imshow(colored)
plt.title("Leaf Disease Interpretation Map")
plt.grid(color='black', linewidth=0.3)
plt.show()
```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image = cv2.imread("/content/WhatsApp Image 2026-02-23 at 2.37.52 PM (1).jpeg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# ----- Disease Detection Kernel -----
kernel = np.array([
    [-1,-1,-1],
    [-1, 8,-1],
    [-1,-1,-1]
])

# ----- Convolution + ReLU -----
h, w = gray.shape
output = np.zeros((h-2, w-2))

for i in range(h-2):
    for j in range(w-2):
        patch = gray[i:i+3, j:j+3]
        value = np.sum(patch * kernel)
        value = max(0, value) # ReLU
        output[i, j] = value

# ----- Normalize -----
if output.max() != 0:
    output = (output/output.max())*255

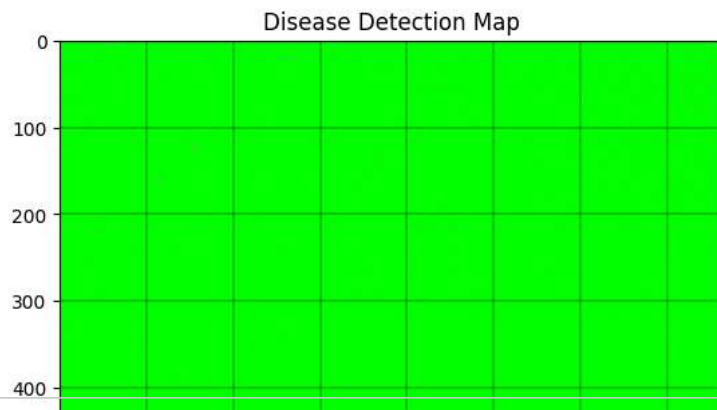
output = output.astype(np.uint8)

# ----- Disease Mapping -----
colored = np.zeros((output.shape[0], output.shape[1], 3), dtype=np.uint8)

threshold = 100 # adjust sensitivity

for i in range(output.shape[0]):
    for j in range(output.shape[1]):
        if output[i,j] > threshold:
            colored[i,j] = [160,160,160] # Grey = Disease
        else:
            colored[i,j] = [0,255,0] # Green = Healthy

# ----- Display -----
plt.imshow(colored)
plt.title("Disease Detection Map")
plt.grid(color='black', linewidth=0.3)
plt.show()
```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("/content/WhatsApp Image 2026-02-23 at 2.37.52 PM (1).jpeg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)

green_lower = np.array([35, 40, 40])
green_upper = np.array([85, 255, 255])

yellow_lower = np.array([20, 80, 80])
yellow_upper = np.array([35, 255, 255])

brown_lower = np.array([10, 50, 20])
brown_upper = np.array([20, 255, 200])
```